Ministère de l'Enseignement Supérieur,
de la Recherche Scientifique et de la Technologie
Université de la Manouba

Ecole Nationale des Sciences de l'Informatique




RAPPORT
De Mémoire de Fin d'Etudes

Présenté en vu de l'obtention du titre
D'INGENIEUR EN INFORMATIQUE

**Par:**
MIMITA Walid

---

Sujet :

# Mise en place d'un back Office d'un intermédiaire en bourse

---

Organisme :CybEx Solutions
Encadrant : MR.BOUZAYANI MOURAD, Chef de projet
Adresse : A46 Résidence Océane, Centre Urbain Nord, Tunis.
Téléphone : 71 28 12 28

CybeX
www.cybexsolutions.net

Année Universitaire 2012/2013

## *Appréciations des encadrants*

MR.BOUZAYANI MOURAD

**(CYBEX)**

MR.BOUAFOURA MOHAMED KARIM

**(ENSI)**

# Remerciements

C’est avec un grand plaisir que je dédie cet ouvrage pour exprimer ma profonde reconnaissance à tous ceux qui nous ont aidés de près ou de loin à la réalisation de ce travail.

Nous tenons à remercier M. MOUSSA Faouzi, Directeur général de Cybex Solution de nous avoir si aimablement accueilli dans son entreprise, ainsi que mes encadreurs à CybEx solutions, à savoir, M. BOUZAYANI MOURAD et M. BEN AMMARA Issam, pour leur encadrement, leurs disponibilités, leurs suivis et les précieux conseils qu’ils nous ont prodigués tout au long du stage.

Nous remercions, également, notre superviseur à l’ENSI M. BOUAFOURA MOHAMED KARIM qui a bien voulu assurer la direction de ce travail. Nous le remercions infiniment pour sa patience, son assistance et ses précieuses recommandations.

Notre dernier mot s’adresse à tous les membres du jury pour l’honneur qu’ils nous font de participer à l’examen de notre mémoire, sans oublier tous nos professeurs à l’ENSI pour la formation qu’ils nous ont donnés.

**ملخّص :**

العمل المقدم في هذه الأطروحة هو المساهمة في إنشاء و تطوير نظام لإدارة وسيط في البورصة وخاصة تطوير عنصر المكتب الخلفي،إستنادا الى مبادئ الهندسة ذاتَ توجه خدماتي و مبادئ تصاميم يحركها المجال .

# Résumé

Le travail élaboré dans cette mémoire porte sur une contribution à la mise en place d'un système de gestion d'un intermédiaire en bourse et plus particulièrement au développement du volet Back-Office tout en basons sur les principes de l'architecture orientée service et du Domain-Driven-Design.

Mots-clés : Bourse, Système de gestion, intermédiaire, SOA, DDD, .NET, RIA.

# Abstract

The work developed in this memory is a contribution to the establishment of a system for managing a broker and particularly the development of back-office component while basing on the principles of service-oriented architecture and the Domain-Driven-Design concepts.

Keywords : stock market, management system, SOA, DDD, .NET, RIA.

# Table des matières

**Introduction générale** **1**

**1 Étude préliminaire** **3**

1.1 Introduction . . . . . 3

1.2 Présentation de l'organisme d'accueil . . . . . 3

1.2.1 Aperçu . . . . . 3

1.2.2 Produits de CybEx Solutions . . . . . 4

1.3 La bourse et les marchés financiers . . . . . 5

1.3.1 Définition . . . . . 5

1.3.2 Les valeurs mobilières . . . . . 5

1.3.3 Ordre de bourse . . . . . 6

1.3.4 La validité des ordres . . . . . 6

1.3.5 Les conditions de prix des ordres . . . . . 7

1.3.6 Les types de client . . . . . 8

1.3.7 Liquidité . . . . . 8

1.3.8 Portefeuille boursier . . . . . 8

1.3.9 Ordre de paiement . . . . . 8

1.4 Domain Driven Design . . . . . 9

1.4.1 Besoin de comprendre le domaine métier . . . . . 9

1.4.2 Le langage omniprésent «Ubiquitous language» . . . . . 10

1.4.3 Concevoir le modèle métier . . . . . 10

1.4.4 Conception dirigée par le modèle . . . . . 10

1.4.5 Les apports du Domain Driven Design . . . . . 13

1.5 Processus de développement . . . . . 13
1.5.1 La méthode 2TUP . . . . . 14
1.6 Etude préalable . . . . . 15
1.6.1 Etude de l'existant . . . . . 15
1.6.2 Critique de l'existant . . . . . 15
1.6.3 Solutions proposées . . . . . 16
1.7 Conclusion . . . . . 16

**2 Capture des besoins 18**
2.1 Introduction . . . . . 18
2.2 Capture des besoins fonctionnels . . . . . 18
2.2.1 Identification des acteurs . . . . . 19
2.2.2 Identification des cas d'utilisations . . . . . 19
2.2.3 Diagrammes de cas d'utilisation . . . . . 20
2.2.4 Description détaillée des cas d'utilisations . . . . . 24
2.2.5 Besoins non fonctionnels . . . . . 30
2.2.6 Identification des classes candidates . . . . . 30
2.3 Capture des besoins techniques . . . . . 30
2.3.1 Etude architecturale . . . . . 31
2.3.2 Etude technologique . . . . . 32
2.4 Conclusion . . . . . 34

**3 Analyse 35**
3.1 Introduction . . . . . 35
3.2 Découpage en catégories . . . . . 36
3.2.1 Dépendance entre les catégories . . . . . 36
3.3 Développement du modèle statique . . . . . 36
3.3.1 Modèle structurel de la catégorie Client . . . . . 37
3.3.2 Modèle structurel de la catégorie Ordre . . . . . 38
3.3.3 Modèle structurel de la catégorie Exécution . . . . . 39
3.3.4 Modèle structurel de la catégorie Liquidité . . . . . 40

3.4 Développement du modèle dynamique . . . . . 41
3.4.1 Cas d'utilisation : Gérer les ordres . . . . . 41
3.4.2 Cas d'utilisation : Gérer l'exécution manuelle . . . . . 45
3.5 Conclusion . . . . . 47

**4 Conception** **48**
4.1 Introduction . . . . . 48
4.2 Conception générique . . . . . 49
4.2.1 Framework techniques . . . . . 49
4.2.2 Les patrons de conception . . . . . 49
4.3 Conception préliminaire . . . . . 52
4.3.1 Conception du modèle logique . . . . . 52
4.3.2 Conception de la structure de présentation . . . . . 54
4.4 Conception détaillée . . . . . 56
4.5 Conclusion . . . . . 64

**5 Réalisation** **65**
5.1 Introduction . . . . . 65
5.2 Environnement matériel et logiciel . . . . . 65
5.2.1 Environnement matériel . . . . . 66
5.2.2 Environnement logiciel . . . . . 66
5.3 Travail réalisé . . . . . 67
5.3.1 Préparation du projet . . . . . 67
5.3.2 Interfaces Homme/Machine . . . . . 68
5.3.3 L'authentification . . . . . 68
5.3.4 Module de gestion des valeurs . . . . . 69
5.3.5 Module de gestion des contrats . . . . . 69
5.3.6 Module de gestion des clients . . . . . 70
5.3.7 Module de gestion des ordres . . . . . 71
5.3.8 Exécution mannuelle . . . . . 73
5.3.9 Module de gestion des liquidités . . . . . 73
5.4 Conclusion . . . . . 75

**Conclusion générale** **76**

# Table des figures


1.1 Logo de « CybEx Solutions » . . . . . 3
1.2 Architecture en couches . . . . . 11
1.3 Pattrons utilisés en conception dirigée par le modèle. . . . . 12
1.4 Processus 2TUP . . . . . 14

2.1 Capture des besoins fonctionnels . . . . . 18
2.2 Diagramme de cas d'utilisation globale . . . . . 21
2.3 Diagramme de cas d'utilisation « gérer les référentiels » . . . . . 22
2.4 Diagramme de cas d'utilisation « Gérer les rôles » . . . . . 23
2.5 Diagramme de cas d'utilisation « gérer les ordres et l'éxecution manuelle » . . 24
2.6 Diagramme d'activités éxecution manuelle . . . . . 29
2.7 Capture des besoins techniques . . . . . 30
2.8 Architecture DDDD . . . . . 32
2.9 Architecture logicielle du WCF . . . . . 34

3.1 L'analyse . . . . . 35
3.2 Découpage en catégories . . . . . 36
3.3 Dépendance entre les catégories . . . . . 36
3.4 Diagramme de classe préliminaire de la catégorie client . . . . . 37
3.5 Diagramme de classe préliminaire de la catégorie ordre . . . . . 38
3.6 Diagramme de classe préliminaire de la catégorie Exécution . . . . . 39
3.7 Diagramme de classe préliminaire de la catégorie Exécution . . . . . 40
3.8 Diagrammede séquence ajout d'un ordre . . . . . 43
3.9 Diagramme de communication VerifierStock . . . . . 44

3.10 Diagramme de séquence exécution manuelle . . . . . 46

4.1 Conception . . . . . 48
4.2 Design Pattern « Specification » . . . . . 50
4.3 Les différents blocs du Design Pattern «MVVM » . . . . . 51
4.4 Rôles des différents composants du Design Pattern «MVVM» . . . . . 52
4.5 Architecture du système de gestion d'un intermédiaire en bourse. . . . . 53
4.6 Maquette du formulaire d'ajout d'ordre . . . . . 55
4.7 Maquette du formulaire de virement interne . . . . . 55
4.8 Maquette d'exécution manuelle . . . . . 56
4.9 Diagramme de classe «ClientBoundedContext» . . . . . 57
4.10 Diagramme de classe «OrdreBoundedContext» . . . . . 59
4.11 Diagramme de classe «ExecutionBoundedContext» . . . . . 61
4.12 Diagramme de classe «LiquiditeBoundedContext» . . . . . 63

5.1 Réalisation . . . . . 65
5.2 Authentification . . . . . 68
5.3 Module Valeurs . . . . . 69
5.4 Module Contrats . . . . . 70
5.5 Module Client . . . . . 70
5.6 Affectation dans un Groupe . . . . . 71
5.7 Tableau de bord . . . . . 71
5.8 Carnet d'ordres . . . . . 72
5.9 Formulaire d'ajout d'un ordre . . . . . 72
5.10 L'execution mannuelle . . . . . 73
5.11 Saisie de Liquidité . . . . . 74
5.12 Ordre de Payement . . . . . 74
5.13 Import Excel . . . . . 75

# Liste des tableaux

1.1 Synthèse des processus de développement . . . . . 13

2.1 Cas d'utilisation . . . . . 20
2.2 Contraintes non fonctionnelles . . . . . 30

4.1 Choix techniques . . . . . 49
4.2 Tableau descriptif des classes de ClientBoundedContext . . . . . 58
4.3 Tableau descriptif des classes de OrdreBoundedContext . . . . . 60
4.4 Tableau descriptif des classes de ExecutionBoundedContext . . . . . 62
4.5 Tableau descriptif des classes de LiquiditeBoundedContext . . . . . 64

5.1 Environnement matériel . . . . . 66

# Introduction générale

Depuis sa création, la bourse n'a cessé d'avoir un rôle majeur dans l'équilibre économique des pays. Considérée comme un lieu de rencontre entre les émetteurs des titres capitaux et les investisseurs, la bourse constitue un marché de valeurs caractérisé par la diversité de l'offre. Néomoins, elle présente aussi bien des taux élevés de perte que de porfit.

À l'origine, la bourse était un marché supposé ouvert à quiconque souhaitant vendre ou acheter des valeurs mobilières. Cependant, afin d'assurer un meilleur contrôle sur le marché, la création d'un organisme officiel pour homologuer les contrats d'achat ou de vente est devenue une nécessité. C'est ainsi que les agents autorisés à exercer en bourse furent limitées à certaines catégories d'opérateurs, ce sont les sociétés d'intermédiation en bourse. En effet, un particulier ne peut plus intervenir directement sur le marché, mais il doit obligatoirement transmettre ses ordres à un intermédiaire financier.

En Tunisie, la profession de l'intermédiation en bourse est soumise à une réglementation stricte régie par la loi **94 - 117 du 14 novembre 1994** et le décret **99 - 2478 du 1er novembre 1999 portant statut des intermédiaires en bourse** qui déterminent les opérations permises, telles que l'exclusivité de la négociation et l'enregistrement en bourse des valeurs mobilières et des produits financiers, la gestion individuelle ou collective des valeurs mobilières, la gestion des portefeuilles et le conseil financer. Ainsi, une bonne gestion des sociétés d'intermédiation en bourse nécessite un système d'information robuste, fiable et conforme à la loi, pour pouvoir traiter, transporter et diffuser une quantité énorme d'informations et de données sensibles.

C'est dans ce contexte que s'inscrit ce Projet de Fin d'Etudes qui consiste à une contribution à la mise en place d'un système de gestion d'un intermédiaire en bourse et plus particulièrement au développement du volet Back Office.

Le présent manuscrit s'articule autour de cinq chapitres, Le premier consiste à introduire le cadre général du projet à travers une étude préliminaire qui consiste à présenter les termes clés du projet et ce, par une introduction au monde de la bourse, une exposition des concepts de

Domaine Driven Design ainsi qu'une présentation du processus de développement utilisé. Le second chapitre expose les besoins fonctionnels et techniques afin de donner un appui au quatrième chapitre qui décrit la phase d'analyse. Le cinquième chapitre traite la conception de la solution. Enfin le dernier chapitre illustre le résultat obtenu.

Chapitre 1

# Étude préliminaire

## 1.1 Introduction

Ce premier chapitre sert, dans une première partie, à présenter l'environnement du stage de fin d'étude et ce, par un aperçu sur le domaine d'activité de la société accueillante «CybEx solutions», ainsi qu'une brève exposition des produits et services offerts par cette dernière.

La seconde partie s'intéresse à la définition des éléments nécessaires pour entamer notre travail, notamment les divers concepts introduits par le domaine de la bourse, le principe de conception suivit, ainsi que la méthode de développement adoptée.

La troisième et la dernière partie s'intéresse à poser la problématique du projet à travers une étude, puis une critique de l'existant afin d'en tirer les solutions adéquates.

## 1.2 Présentation de l'organisme d'accueil

L'organisme d'accueil est une entreprise tunisienne exerçant dans le domaine de développement des solutions financières.

FIGURE 1.1 – Logo de « CybEx Solutions »

### 1.2.1 Aperçu

CybEx est une société de services et de développement des solutions Front Office, Middle Office et Back Office pour les métiers de bourse. Bâtit sur le savoir et le savoir-faire des professionnels des métiers de la bourse, bénéficiant d'une solide expérience dans le domaine financier.

Outre les missions de conseil, l'audit et la consultation auprès des banques et des intermédiaires en bourse, CybEx offre des solutions expertes et innovantes pour les intermédiaires en bourse, les banques, les sociétés de gestion et les établissements financiers spécialisés.

### 1.2.2 Produits de CybEx Solutions

CybEx Solutions offre un ensemble de produits particulièrement adaptés aux besoins des sociétés d'intermédiation en bourse aussi que pour leurs clients et partenaires. Ces solutions peuvent être défalquées en deux catégories, des solutions client adressés effectivement aux clients de l'intermédiaire en bourse et des solutions Pro dédiées aux professionnels de l'intermédiaire à savoir les négociateurs, les commerciaux et les agents de Back Office.

- **Solutions Clients :**

Dans ce volet, « CybEx Solutions» offre une panoplie de solutions parmi lesquelles ou peut citer :

**« CybEx Mobile Trading »** qui est une solution mobile permettant la consultation du marché boursier Tunisien et le passage d'ordres en temps réel à travers un Smartphone.

**« CybExETrade »** qui est une solution web, permettant la réception, en temps réel, du flux boursier complet (meilleures limites, profondeur, intraday, quantités, cours, limites, news, etc.), ainsi que la gestion complète d'un carnet d'ordres. Le client pourra ainsi envoyer, modifier et annuler ses ordres d'achat et de vente.

**« CybExPluvalue et Impôt »** qui est une solution web du type Application Intenet Riche, permettant au client à travers un accès sécurisé sur internet, mais également à l'intermédiaire en bourse, de calculer la Plus-value réalisée pour une période donnée ainsi que le montant exacte des impôts à déclarer.

- **Solutions Pro :**

Dans ce context, « CybEx Solutions» offre une ensemble de solutions parmi lesquelles ou peut citer :

**« CybExMarketFeed »** qui est une solution de consultation assurant l'acheminement, aux opérateurs du marché, de l'ensemble des flux nécessaires à l'exercice de leur activité et à l'optimisation de leur processus de prise de décisions. Le serveur CybEx MARKET FEED garantie à l'intermédiaire l'autonomie et la disponibilité de l'ensemble des données du marché et ce à toutes heures. Cette solution est destinée aux professionnels de la bourse comme aux négociateurs ou les analystes financiers œuvrant au sein d'un réseau d'entreprise (LAN/VPN).

**« CybEx OMS »** est une solution à travers laquelle, CybEx fut la première société à avoir passé avec succès les tests de validation de la BVMT et du CMF de son logiciel CybEx

OMS® (Order Management System) et ce en Janvier 2009. CybEx OMS s'adresse à tous ceux qui souhaitent sécuriser la gestion et l'envoi de leurs ordres en temps réel selon des règles et des contraintes prédéfinies.

**« CybEx CEA »** qui est une solution de Gestion efficace et optimisée des Comptes Epargnes Actions. CybEx CEA permet la (i) consultation de la liste détaillées des clients CEA, (ii) détermination des clients qui ont des soldes critiques, (iii) détermination et transfert des droits, (iv) détermination et transfert des dividendes, (v) Détermination et transfert des plus-values réalisées, (vi) transfert multiple des plus-values réalisées par PF ou par Ligne.

## 1.3 La bourse et les marchés financiers

### 1.3.1 Définition

La bourse est le lieu ou les investisseurs peuvent acheter et vendre des titres de capital ou de créance émisent par l'état des entreprises, ou même des collectivités locales. Son rôle principal est d'assurer la liquidité des titres détenus par les investisseurs. Cette liquidité permet aux émetteurs de se procurer des fonds pour financer leur croissance en faisant appel au public.A ce titre, la bourse constitue l'une des sources principales du financement de l'économie.

La bourse des valeurs mobilières de Tunis (BVMT) [5] gère le marché Tunisien des valeurs mobilières par :

- l'admission de nouveaux titres à la cote de la bourse.
- l'organisation des échanges et la cotation des titres dans les meilleures conditions d'égalité, de sécurité et de transparence.
- la diffusion des informations boursières.

### 1.3.2 Les valeurs mobilières

Les valeurs mobilières sont des titres négociables et interchangeables, ils peuvent être cotés en bourse ou non. Les deux grandes catégories de valeurs mobilières sont les actions et les obligations,mais il en existe d'autres, telles que les options, les bons de souscription et les Warrants.

#### Les actions

Les actions [6] sont des titres de propriété d'une société. Chaque action représente une fraction du capital de cette entreprise. La possession d'actions donne des droits à leur détenteur auprès

de l'entreprise concernée ; droits liés au statut de copropriétaire de l'entreprise et qui s'exercent proportionnellement au nombre d'actions détenues.

#### Les obligations

En cas de besoin de financement, la bourse constitue pour les entreprises, privées ou publiques, une alternative au crédit bancaire. Dans ce cas, l'entreprise émet des titres de créance appelés obligations[6].

Chaque obligation représente une fraction d'un emprunt émise par l'entreprise sur le marché boursier. Le porteur de l'obligation devient donc créancier de l'entreprise.

Le porteur d'obligation, court un risque inférieur au porteur d'action, car l'obligation lui assure des garanties de rémunération et de remboursement. En revanche, le porteur d'obligation ne joue pas des mêmes droits que le porteur d'action : il ne bénéficie ni du droit sur les bénéfices, ni du droit de gestion ou du droit de vote. Les obligations sont rémunérées en fonction d'un taux d'intérêt sur la valeur faciale. Ce taux est fixé par voie contractuelle dès l'émission du titre de créance.

### 1.3.3 Ordre de bourse

Un ordre de bourse est un ordre passé par une personne physique ou morale à son intermédiaire pour acheter ou vendre en bourse. Cet ordre comporte obligatoirement :

- Le sens de l'opération : achat ou vente.
- La nature des titres : code ISIN [1]de la valeur.
- Le nombre de titres : quantité.
- Le client.
- La validité : jour, révocation ou à date déterminée.
- Les conditions de prix :à tout prix , à cours limité ou autres modalités.

### 1.3.4 La validité des ordres

Les ordres sur la bourse Tunisienne peuvent avoir différentes options de validité tel que :

- La validité jour : l'ordre transmis n'est valable que pour la séance du jour. Non exécuté à l'issue d'une séance, il est échu et nécessite la saisie d'un nouvel ordre.
- La validité révocation : l'ordre ainsi libellé, aura pour échéance le dernier jour de l'année boursière en cours.
- La validité date : le choix d'une date fixe est possible.

1. International Securities Identification Number

### 1.3.5 Les conditions de prix des ordres

Il existe différentes conditions sur le prix d'un ordre :

#### Ordre à cours limité

Ce type de modalité est à la fois le plus simple. Il traduit le prix maximal qu'accepte de payer un acheteur ou le prix minimal qu'exige un vendeur. En cours de séance, ces ordres restent inscrits en carnet jusqu'à ce qu'une contrepartie accepte ces conditions. Il permet de maîtriser le prix d'exécution et donc de se protéger contre les fluctuations de cours. Son exécution peut être partielle.

#### Ordre à tout prix (ATP)

Il s'agit d'un ordre sans limite de prix spécifié. Au moment de l'ouverture, ce type d'ordre est prioritaire sur tous les autres types d'ordres et peut faire l'objet d'une exécution partielle. Cet ordre ne permet pas de maîtriser le cours d'exécution.

#### Ordre à la meilleure limite

Il s'agit d'un ordre sans limite de prix spécifié. En arrivant sur le marché :

- A l'ouverture, il est transformé en ordre limité au cours d'ouverture. Il est exécuté en fonction des soldes disponibles du carnet d'ordres après prise en compte des ordres au marché et des ordres à cours limité.
- En cours de séance, l'ordre est positionné au prix de la meilleure offre en attente (pour un achat) ou de la meilleure demande (pour une vente). En cas d'exécution partielle, la quantité restante se transforme en ordre à cours limité au cours d'exécution.

#### Ordre à seuil de déclenchement (Stop Limit)

Il permet à un investisseur de se porter acheteur ou vendeur à partir d'un cours déterminé par le seuil de déclenchement. Il comporte une limite à partir de laquelle il se transforme en ordre « à tout prix ». Ce type d'ordre ne permet pas de maîtriser le prix d'exécution.

#### Ordre à plage de déclenchement (Stop Loss)

Ce type d'ordre permet de se porter acheteur ou vendeur à partir d'un cours déterminé par le seuil de déclenchement, tout en se fixant une limite maximale pour un ordre d'achat ou minimale pour un ordre de vente. Il permet la maîtrise du prix d'exécution et permet de se protéger contre d'éventuels renversements de tendance.

**Ordre au cours d'ouverture**

Ordre réservé aux valeurs les plus liquides donnant accès à une phase de cotation spécifique entre 9h00 et 10h00. Il ne peut être assorti que d'une validité « jour ». Il permet de connaître par avance le cours d'exécution, sous réserve de cotation de la valeur et d'existence de quantité suffisante.

### 1.3.6 Les types de client

La bouse Tunisienne prend en charge les types de client suivant :

- Libre de nationalité Tunisienne.
- Géré de nationalité Tunisienne.
- Teneur de marché.
- Compte propre.
- Libre de nationalité étrangère.
- Géré de nationalité étrangère.
- Compte OPCVM [2].

### 1.3.7 Liquidité

La liquidité [7] est la capacité et la rapidité avec laquelle il est possible d'acheter ou de vendre un actif ou un passif sur le marché sans que les prix du support n'en soient fortement affectés.

### 1.3.8 Portefeuille boursier

Le portefeuille boursier est la représentation de l'ensemble des titres sur lesquels un agent économique a investi sur le marché financier.

### 1.3.9 Ordre de paiement

Un ordre de paiement [7] est une instruction donnée par un client à son teneur de compte, de mettre une somme d'argent à la disposition d'un bénéficiaire.

2. Organisme de Placement Collectif en Valeurs Mobilières

# Remarque

Vue la délicatesse des différentes notions définies dans ce qui précède, il serait difficile à un développeur de côtoyer le domaine de la bourse dans un temps réduit pour y proposer des solutions. Dans ce sens, il serait fort intéressant de chercher à simplifier la tâche de l'informaticien d'une part et du financier d'autre part lors de la réalisation du logiciel où l'intervention de ces deux experts est indispensable.

La partie suivante introduit une méthode de conception simplificatrice et générique, qui à notre avis adéquate pour notre travail.

## 1.4 Domain Driven Design

Domain Driven Design [1], noté en litérature DDD, ou conception pilotée par le domaine est une approche basée sur un ensemble de principes et de modèles, qui combine les bonnes pratiques de conception et d'implémentation pour permettre aux développeurs logiciels de créer et concevoir des applications performantes pour des domaines métiers complexes et délicats à apprendre

La conception pilotée par le domaine n'impose aucune technologie, ni méthodologie mais elle donne la priorité au domaine métier face aux aspects techniques pour diriger le développement.

### 1.4.1 Besoin de comprendre le domaine métier

Afin de concevoir un logiciel qui répond aux besoins des utilisateurs, il est primordial de comprendre le domaine métier dans lequel le système va opérer.

Une bonne connaissance du domaine consiste à la création d'un modèle représentatif en faisant travailler les spécialistes informatiques avec des experts métier. Cette étape est nécessaire et indispensable pour aboutir à un logiciel qui reflète le domaine.

En pratique, cette approche se heurte à quelques difficultés dues à des problèmes de communication. Chaque membre du projet à savoir développeur, architecte, chef de projet ou expert métier est inscrit dans un domaine propre à lui. Chacun possède son propre jargon et ses propres termes qui peuvent être incompréhensibles par les autres membres.

Afin de remédier a ce problème, la DDD propose d'établir un langage commun, appelé «Ubiquitous Language» ou langage omniprésent, qui rassemble tous les termes et mots clés, et qui sera utilisé tout le long du processus de développement du logiciel.

### 1.4.2 Le langage omniprésent «Ubiquitous language»

L'un des principes de base de la DDD, est l'établissement d'un langage commun entre les différents intervenants dans le processus de développement.

Pour éviter l'incompréhension entre développeurs, architectes et experts métier, ce langage introduit donc des concepts techniques, architecturals et métiers intégrés ensemble afin que les contraintes soient prises en compte aussi bien par les spécialistes métiers que par les développeurs afin de produire un logiciel robuste qui répond aux besoins des utilisateurs.

### 1.4.3 Concevoir le modèle métier

Selon Eric Evans le fondateur du la DDD, un modèle du domaine, n'est pas un diagramme particulier mais c'est l'idée qu'on cherche à dévoiler à travers. Il ne représente pas forcément la connaissance contenue dans le cerveau d'un professionnel du domaine mais, une abstraction rigoureusement sélective et organisée de cette connaissance.

Afin de concevoir un modèle de domaine le processus peut s'effectuer à travers des ateliers de travail «workshops» qui réunissent des spécialistes métiers, architectes et développeurs en considérant le langage omniprésent comme colonne vertébrale du modèle à réaliser.

Il est important d'impliquer les développeurs et les experts techniques dans le processus d'établissement du modèle afin d'aboutir à un modèle qui puisse être exprimé convenablement dans les phases de conception et d'implémentation.

les développeurs apportent le "feedback" nécessaire pour s'assurer que le modèle porposé par expert métier peut être implémentable en logiciel.

### 1.4.4 Conception dirigée par le modèle

L'obtention d'un bon modèle qui reflète les concepts de domaine métier et qui représente précisément les besoins des utilisateurs est extrêmement important, mais un échec dans la phase de transformation du modèle de domaine en code peut mener à un logiciel de qualité discutable.

Le passage du modèle de domaine à l'implémentation est souvent délicate.

Afin de donner un support solide à la conception et pour remédier aux problèmes d'architecture, la DDD introduit un ensemble de concepts dans le langage commun. Ces notions doivent être assimilées par les développeurs, les architectes et les spécialistes métiers.

#### 1.4.4.1 Architecture en couches

Le Domain Driven Design renforce le paradigme d'architecture en couches et propose une architecture composée de quatre couches séparées dont chacune est cohérente et ne dépend

que de la couche inférieure afin de fournir un couplage faible entre elles. Toute la logique du domaine métier est donc encapsulée dans une seule couche ce que, en résulte une coucheapplicative fine responsable de stocker l'état de l'interface utilisateur et de communiquer avec la couche domaine

De plus une couche technique « infrastructure transverse » dénuée de toute logique métier, qui a pour rôle de fournir un support solide à l'application dans laquelle seront disponibles toutes les fonctionnalités facilitant la gestion des journaux d'exécution (logs),la sécurité, la gestion de cache etc., ce qui améliore la robustesse et les performances de l'application (cache, montée en charge, gestion des threads, etc.).

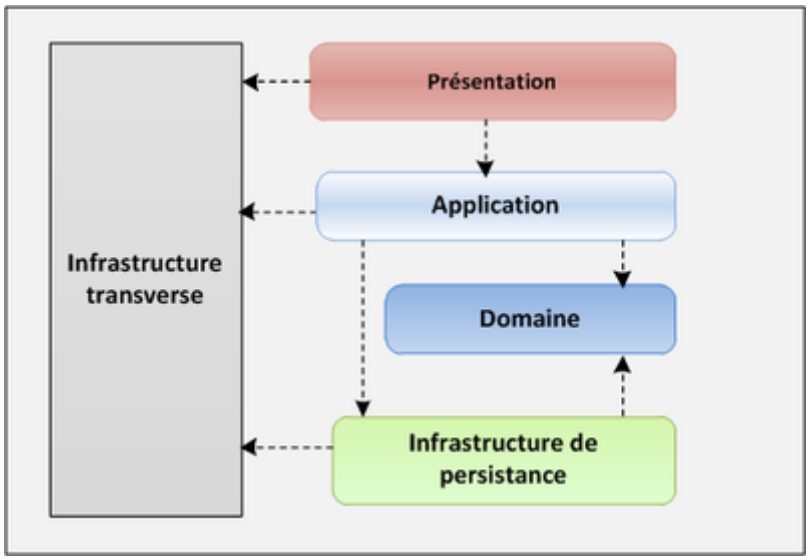



FIGURE 1.2 – Architecture en couches

#### 1.4.4.2 Les Entités (Entities)

Une entité est un objet ayant une identité unique qui reste constante tout au long du cycle de vie de l'application.

#### 1.4.4.3 Les objets valeurs (ValueObject)

Les objets-valeurs viennent complémenter les entités. Ils découlent de deux concepts importants : ne pas avoir d'identité et être partageable par plusieurs modules métiers.

#### 1.4.4.4 Les Services

Un service définit une opération ou une activité bien déterminée qui n'appartient à aucun objet du modèle. Ces opérations touchent plusieurs objets et assurent la coordination entre eux.

#### 1.4.4.5 Les Agrégats (Aggregates)

Un agrégat est un ensemble d'objets liés entre eux par une relation logique et formant une seule unité. Il fait la séparation entre ces objets internes et les objets externes. Chaque agrégat est défini par une seule racine, qui doit être une entité et est le seul objet accessible de l'extérieur contenant les références des différents objets de l'agrégat.

#### 1.4.4.6 Les Fabriques

Les fabriques (Factories) sont utilisées pour la création d'objets complexes dont on doit savoir la structure interne et les règles appliquées en respectant le principe d'encapsulation des objets. Ils sont très utiles dans la création des agrégats.

#### 1.4.4.7 Les entrepôts (Repositories)

L'Entrepôt ont pour rôle de conserver des références vers des objets. Quand un objet est créé, il peut être sauvegardé dans l'Entrepôt, et y être récupéré ultérieurement en cas de besoin. Ils peuvent être un intermédiaire entre les objets du domaine et l'infrastructure.

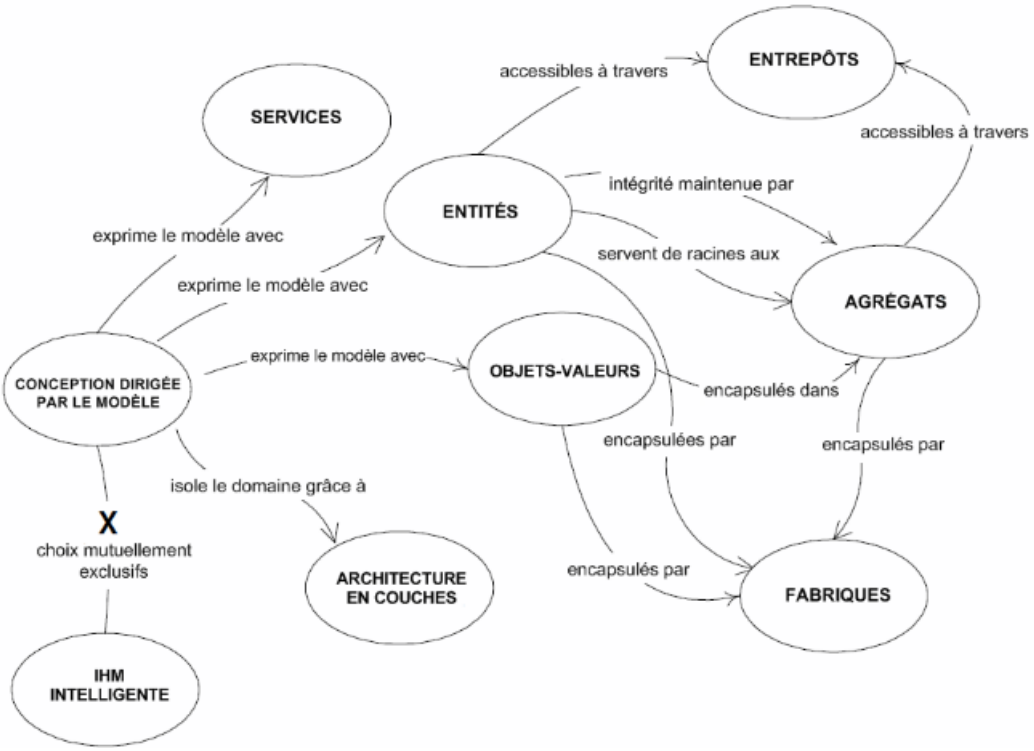



FIGURE 1.3 – Pattrons utilisés en conception dirigée par le modèle.

Le diagramme [1] de la figure1.2 montre les différents pattrons utilisés en conception dirigée par le modèle et des relations qui existent entre eux.

### 1.4.5 Les apports du Domain Driven Design

Le Domain Driven Design, n'impose aucun formalisme, ni méthode pour comprendre et acquérir le domaine. Il insiste simplement sur l'importance d'une communication continue entre tous les membres du projet en utilisant un langage partagé et unifier tout au long du projet.

De point de vue architecture, la concentration du métier dans une seule couche rend le code plus maintenable et évolutif. Un autre point important est l'isolation du code métier. Ceci facilite le développement des tests unitaires qui permettent ainsi de contrôler de façon efficace l'évolutivité du code.

## 1.5 Processus de développement

Dans le but d'aboutir à un logiciel de qualité qui répond aux besoins exigés dans des temps et des coûts prévisibles. Le choix du processus de développement convenable est une phase primordiale dans tout projet. Afin d'appliquer une méthode rigoureuse sur la conduite de notre projet, nous avons établie une synthèse des processus de développements les plus recommandés.

Le tableau suivant illustre une étude comparative en dressant les avantage et les inconvénients de Scrum, XP et 2TUP afin de choisir le plus adéquat.

| / | **Description** | **Points Forts** | **Points Faibles** |
|---|---|---|---|
| **Scrum** | -Méthode agile<br>-Découpe le projet en sprint. | -itératif<br>-incrémental<br>-Facile à mettre en œuvre<br>-La plus populaire des méthodes Agile | -Ne couvre aucune technique d'ingénierie du logiciel<br>-Doit être compléter des pratiques de qualité du logiciel |
| **XP**<br>« eXtreme Programming » | -Ensemble de « Bests Practices » de développement.<br>-Cible des projets de moins de 10 personnes | - itératif<br>-simple à mettre en œuvre<br>- fait une large place aux aspects techniques | - ne couvre pas les phases en amont et en aval au développement<br>- assez flou dans sa mise en œuvre |
| **2TUP**<br>« Two Track Unified Process » | -s'articule autour de l'architecture logiciel<br>-propose un cycle de développement en Y<br>- cible des projets de toutes tailles | -itératif<br>- fait une large place à la technologie<br>- définit les profils des intervenants, les plannings, etc. | - plutôt superficiel sur les phases situées en amont et en aval du développement : capture des besoins, maintenance, etc |

TABLE 1.1 – Synthèse des processus de développement

### 1.5.1 La méthode 2TUP

A partir de l'etude menée dans la section précidante, la méthode 2TUP [2] parait la plus adaptée à notre projet car elle s'articule autour de l'architecture, elle fait une large place à la technologie et grâce a sa compatibilité avec les principes de DDD. Elle propose un cycle de développement en Y qui s'articule autour de trois phase essentielles a savoir une branche fonctionnelle, une branche technique et une branche centrale de réalisation et ce l'illustré la figure 1.4.

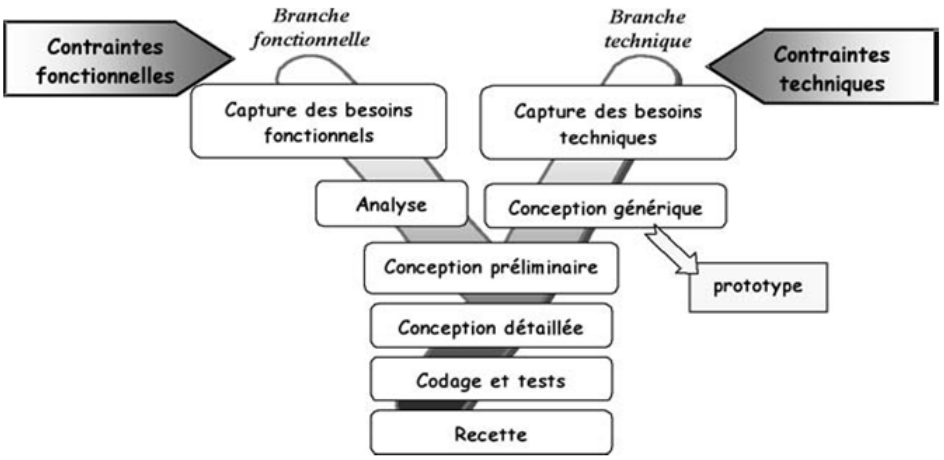



FIGURE 1.4 – Processus 2TUP

#### 1.5.1.1 Une branche fonctionnelle

Dans cette phase nous allons acquérir la connaissance du domaine métier. Cette étape constitue généralement un investissement pour le moyen et le long terme. Les fonctions du système sont en effet indépendantes des technologies utilisées. Cette branche comporte les étapes suivantes :

- La capture des besoins fonctionnels, qui produit un modèle des besoins focalisé sur le métier des utilisateurs.
- L'analyse.

#### 1.5.1.2 Une branche technique

Capitalise un savoir-faire technique. Elle constitue un investissement pour le court et moyen terme. Les techniques développées pour le système peuvent l'être en effet indépendamment des fonctions à réaliser. Cette branche comporte les étapes suivantes :

- La capture des besoins techniques.
- La conception générique.

#### 1.5.1.3 Une branche de réalisation :

la réalisation du système consiste à fusionner les résultats des 2 branches. Cette fusion conduit à l'obtention de la forme Y du processus. Cette branche comporte les étapes suivantes :

- La conception préliminaire.
- La conception détaillée.
- Le codage et le test.
- L'intégration.

## 1.6 Etude préalable

Dans cette partie les problématiques du projet sont exposés par le biais d'une étude puis d'une critique de l'existant.

Par la suite les solutions envisagées sont succinctement présentées.

### 1.6.1 Etude de l'existant

Avec la solution «CybEx-Stock», CybEx Solutions a pu, en un premier temps, commercialiser sur le marché tunisien une solution complète de front-office, middle-office et back-office. Cette solution recouvre tous les modules nécessaires au bon fonctionnement d'une société d'intermédiation en bourse comme la mise en place des référentiels, la gestion des ordres émis et reçus, la gestion des (OPV [3]-OPA [4]), la gestion des OST [5], la gestion des portefeuilles, la gestion de la caisse, l'intégration des SICAV, [6] l'historisation, l'archivage en ligne, le monitoring et le suivi de l'activité des commerciaux.

### 1.6.2 Critique de l'existant

Malgré les avantages offerts par « CybexStock », l'application a montré plusieurs défauts. En effet, cette solution se présente sous forme d'un client lourd développé par la technologie des WinForms, présentant plusieurs inconvénients notamment sur le coût de la maintenance et du déploiement.

De plus, à cause de l'accroissement exponentiel de la quantité de données traitées du à l'augmentation de la taille de base de données utilisé, «Cybex-Stock» commence à montrer

3. Ordre Public de Vente
4. Ordre Public d' Achat
5. Opérations Sur Titre
6. Société d'investissement à capitale variable

certaines limites au niveau de la performance et plus précisément en termes de fiabilité et temps de réponse. Ce qui a conduit à certains cas de disfonctionnement.

D'autre part, l'évolution des besoins des clients a souvent conduit les ingénieurs de « CybEx solutions» à modifier les codes de l'application, parfois sans tenir compte des patrons de conception et du principe de la séparation entre les couches logicielles. Ce qui induit à une complexité notable surtout lors de la maintenance et lors du traitement de quelques besoins spécifiques.

Encore, la méthodologie de développement suivie au cours de la mise en place de l'application « CybEx-Stock » n'était pas bien adaptée au domaine et à la complexité de la solution, ce qui a nettement contribué à la dégradation de la qualité du logiciel.

Afin de pallier aux différents problèmes soulevés, qui ont tout angoissé aussi bien les utilisateurs que l'équipe de maintenance, une refonte de l'application d'un nouveau point de vue, s'avère nécessaire. En Effet, les solutions qui ont été envisagées dans le cadre de ce projet seront détaillées dans ce que suit.

### 1.6.3 Solutions proposées

Afin d'obtenir un logiciel de qualité qui répond aux besoins exigés et qui permet de remédier aux défauts limitant la performance de l'application, le choix est dirigé vers une refonte intégrale de l'application ce qui permettera de contourner d'une manière définitive et efficace les différents problèmes précédemment cités.

A fin de diminuer les coûts de la maintenance et de déploiement, la solution envisagée est une application RIA(Rich Internet Application), qui sera détaillée davantage plus loin dans ce manuscrit.

Pour faire face à la complexité du domaine de l'application et après une étude approfondie sur les dernières recommandations dans le développement des applications pour des domaines similaires, le choix s'est porté sur la mise en place d'une solution qui suit les principes de Domain Driven Design en se basant sur ses apports et sa compatibilité avec l'architecture SOA(Service Oriented Architecture).

## 1.7 Conclusion

Dans ce premier chapitre, le contexte général de stage, les notions de la bourse et des marchés financiers ainsi que les principes "Domain Driven Design" ont été présentés. De plus, une étude préalable où une critique de l'existant nous a conduits à tracer les grandes lignes de la solution à développer.

En se référant aux éléments présentés dans ce chapitre, nous aborderons dans le chapitre suivant la première étape de la méthode 2TUP, à savoir, la capture de besoins fonctionnels et techniques.

Chapitre 2

# Capture des besoins

## 2.1 Introduction

Le choix étant fixé de travailler avec la méthode de développement 2TUP. Ce chapitre sera consacré pour explorer les premières phases des deux branches technique et fonctionnelle du processus à savoir, la capture des besoins.

## 2.2 Capture des besoins fonctionnels

La capture des besoins fonctionnels est la première étape de la branche gauche du cycle en Y, comme l'illustre la figure 2.1. Elle utilise le cahier des charges pour compléter les besoins fonctionnels. Dans ce chapitre, nous allons identifier les acteurs du système, détailler et organiser les cas d'utilisation, via différents diagrammes pour arriver finalement à distinguer les classes candidates de notre système.

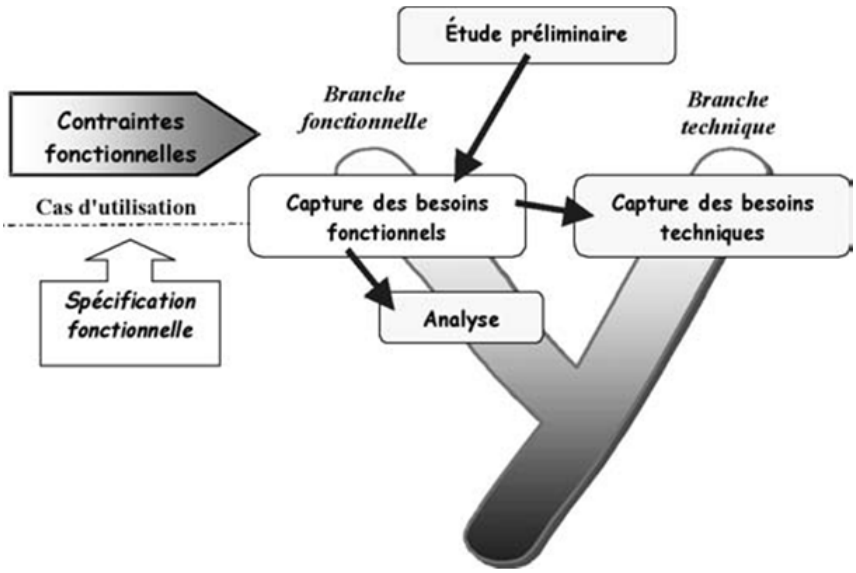



FIGURE 2.1 – Capture des besoins fonctionnels

### 2.2.1 Identification des acteurs

L'identification des acteurs d'un système permet en première phase de délimiter le système et de comprendre le rôle de chaque acteur et de ses exigences. Dans notre application, nous avons identifié les acteurs principaux suivants.

#### 2.2.1.1 L'administrateur

C'est un utilisateur particulier du système, il a tout les privilèges nécessaire pour administrer l'application.

#### 2.2.1.2 Le commercial

C'est une personne chargée de la réception des clients, il peut également envoyer des ordres. Chaque commercial possède ses propres clients. Il ne peut pas traiter les demandes posées par les clients des autres commerciaux. Pour des raisons d'efficacité et de rapidité, les intermédiaires s'autorisent de laisser les commerciaux passer les ordres de leurs clients directement sur le marché.

#### 2.2.1.3 Le superviseur

C'est une personne chargé du contrôle de ses commerciaux. Il peut annuler ou modifier une opération qu'il la juge incorrect. Outre la supervision, il doit bénéficier des mêmes fonctionnalités qu'un commercial. Ainsi, un superviseur peut autoriser, modifier, annuler ou bloquer un ordre passé par un commercial qui ne respecte pas les contraintes prédéfinies dans le système.

#### 2.2.1.4 L'agent de Backoffice

Il est responsable de la gestion des opérations boursières et de la validation des journées comptables.

### 2.2.2 Identification des cas d'utilisations

Dans cette étape nous allons déterminer les fonctionnalités du système de gestion d'un intermédiaire en bourse et assigner les cas d'utilisation aux différents acteurs précédemment identifiés.Le tableau 2.1 recense l'ensemble des cas d'utilisations ainsi que les des messages émissent et reçus par les acteurs de chaque cas.

| **Cas d'utilisation** | **Acteur** | **Message(s) émis/reçus par les acteurs** |
|---|---|---|
| Gérer les ordres | -Commercial<br>-Superviseur | Emet : création, modification, annulation et Filtrage du carnet d'ordres<br>Reçoit : carnet d'ordres |

| | | |
|---|---|---|
| Gérer la caisse | -Agent de Backoffice | Emet : création, annulation et import de liquidité<br>Reçoit : liste des liquidités. |
| Gérer les Ordres de payements | -Commercial | Emet : création, annulation Reçoit : liste des Ordres de payement des comptes visibles pour l'utilisateur. |
| Gérer les comptes/clients | - Agent de Backoffice | Emet : création, modification, blocage<br>Reçoit : liste des comptes/clients visible pour l'utilisateur. |
| Gérer le référentiel des contrats | -Agent de Backoffice | Emet : création, modification, annulation<br>Reçoit : liste des contrats |
| Gérer le référentiel des intermédiaires | -Agent de Backoffice | Emet : création, modification, suppression<br>Reçoit : liste des intermédiaires. |
| Consulter le tableau de bord client. | -Commercial<br>-Agent de Backoffice | Reçoit : tableau de bord Client. |
| Exécution manuelle | -Agent de Backoffice | Emet : contrat, ordre<br>Reçoit : liste des ordres, liste des Contrats. |
| Gérer les rôles | -Administrateur | Emet : consultation, création, modification et suppression des rôles des utilisateurs. Reçoit : liste des utilisateurs. |

TABLE 2.1 – Cas d'utilisation

Aprés avoir identifié les cas d'utilisation et leurs acteurs, nous pouvons maintenant les représenter graphiquement dans des diagrammes de cas d'utilisation.

### 2.2.3 Diagrammes de cas d'utilisation

Les diagrammes de cas d'utilisation décrivent l'interaction entre un utilisateur (humain ou machine) et un système. Ils permettent de recueillir, d'analyser et d'organiser les besoins sous une forme qui facilite la compréhension commune entre développeurs et utilisateurs.

#### 2.2.3.1 Diagramme de cas d'utilisation globale

La figure 2.2 illusre le diagramme de cas d'utilisation globale du systéme.

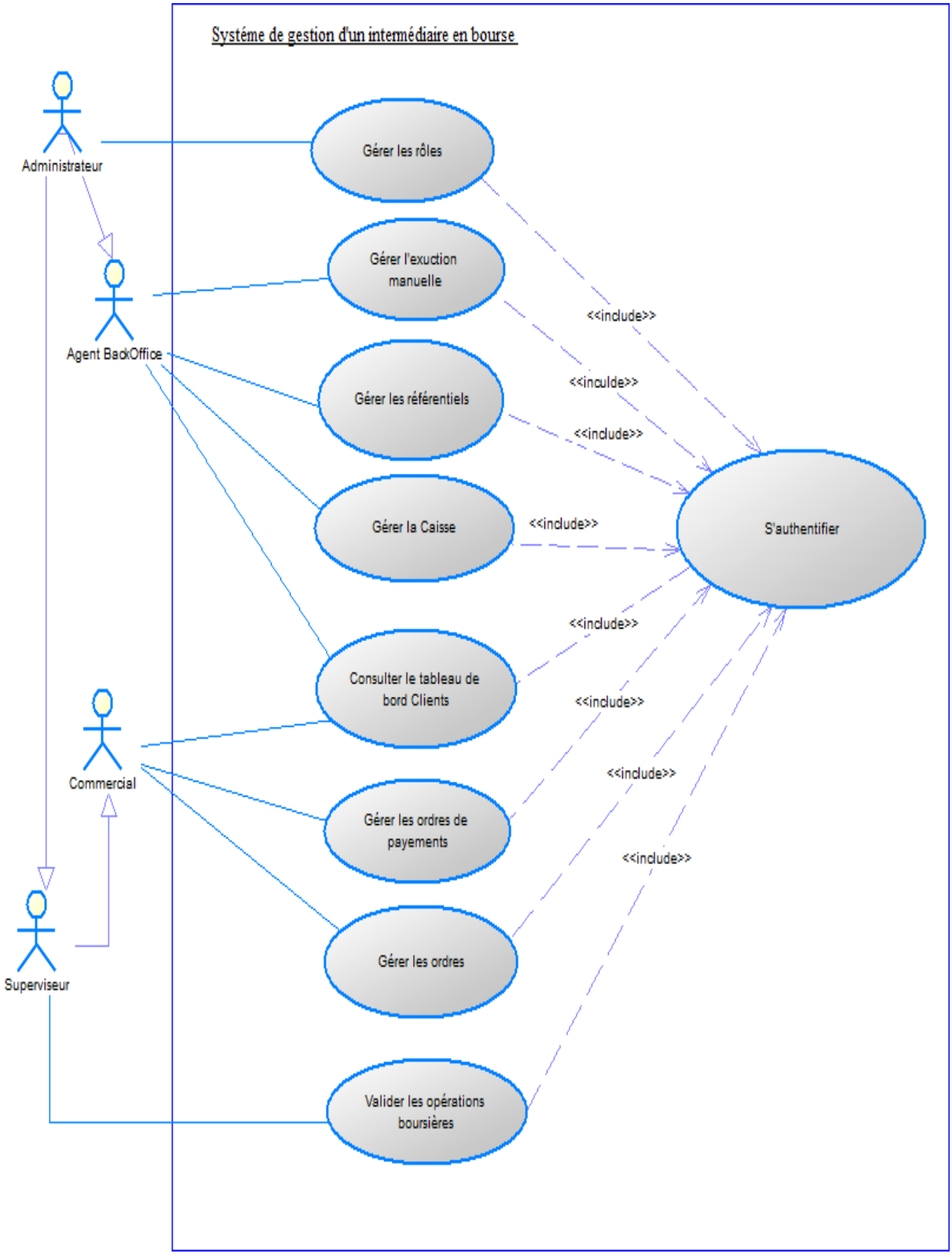



FIGURE 2.2 – Diagramme de cas d'utilisation globale

#### 2.2.3.2 Raffinement des cas d'utilisation

Dans cette phase, nous allons détailler les cas d'utilisation liés au système de gestion d'un intermédiaire en bourse présent dans le diagramme des cas d'utilisation globale dans le but de recenser toutes les interactions entre le système et les acteurs.

**Diagramme de cas d'utilisation « gérer les référentiels »**

La figure 2.3 explique le cas d'utilisation « gérer les référentiels »

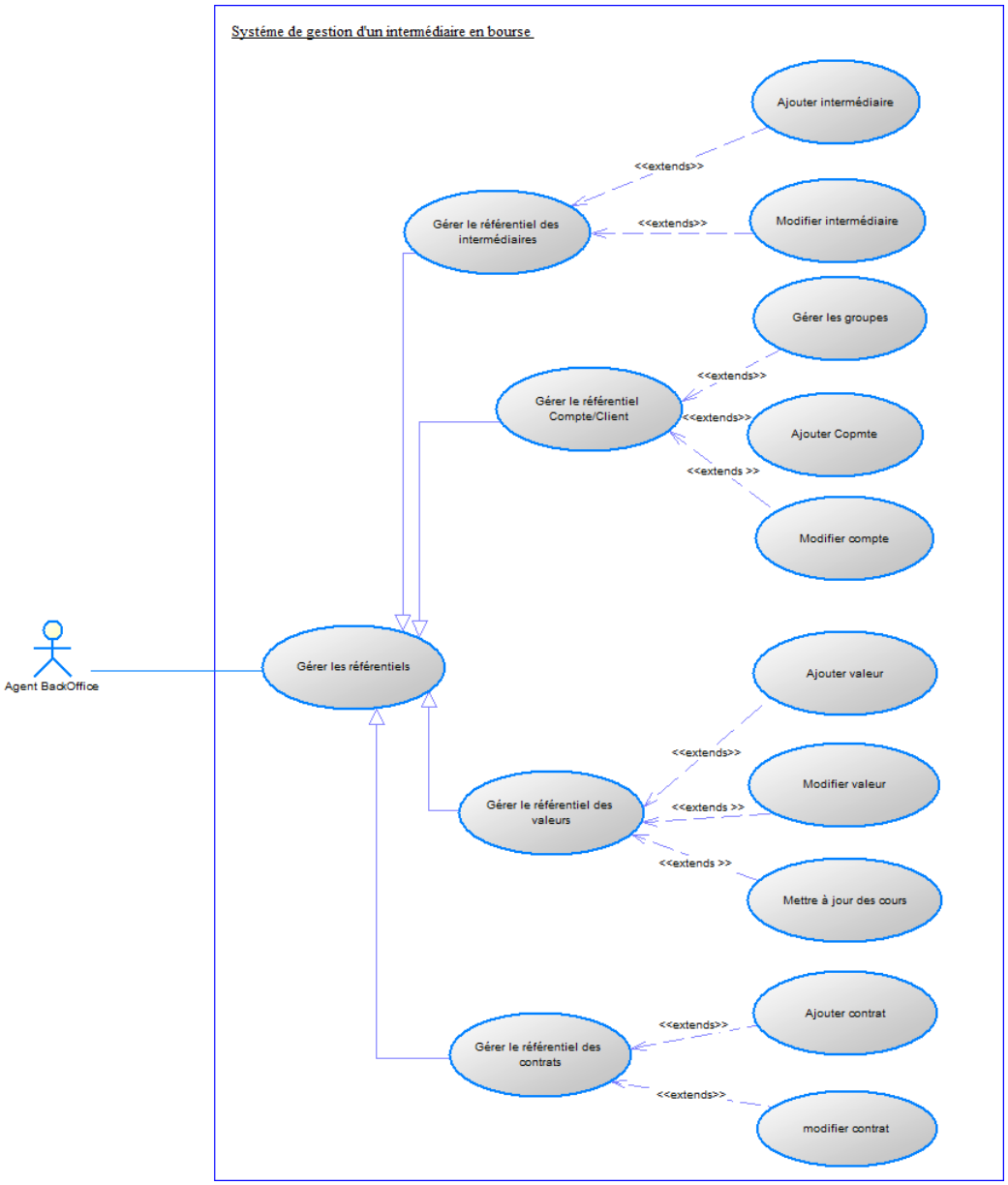



FIGURE 2.3 – Diagramme de cas d'utilisation « gérer les référentiels »

Le système de gestion d'un intermédiaire permet de gérer les référentiels suivants :

- Le référentiel des Clients/Comptes : Il permet de préciser les informations essentielles des clients tel que la gestion (libre, gérée), le type de client (Géré de nationalité Tunisienne, Teneur de marché, Compte propre, Libre de nationalité étrangère, Géré de nationalité étrangère), la commission à appliquer et le commercial qui prendra en charge le client. Aussi, il permet d'associer à un client différents types de comptes (ordinaire, sicav, CEA, FCP, etc.) et d'assembler plusieurs comptes dans un groupe.
- Le référentiel des valeurs : Il propose les fonctions d'ajout et d'édition des valeurs de différents types. Il propose également la fonction de mise à jour automatique des cours en se basent sur le fichier texte délivré par la bourse précisant le cours de clôture de toutes les valeurs ayant été cotées durant la séance.
- Le référentiel des intermédiaires : Il propose les fonctions d'ajout, d'édition et de recherche des intermédiaires.
- Le référentiel des contrats : Il propose la création, l'édition et la recherche d'un contrat d'achat ou de vente. Le contrat sera ensuite exécuté au moyen de l'exécution manuelle.

**Diagramme de cas d'utilisation « gérer les rôles »**

L'administrateur est le seul capable de spécifier les permissions aux différents usagers de l'application. Pour ce faire, il doit auparavant créer un rôle. Ensuite, il peut pour chaque partie de l'application soit autorisée ou bannir l'accès. La figure 2.4 explique le diagramme de cas d'utilisation correspondant.

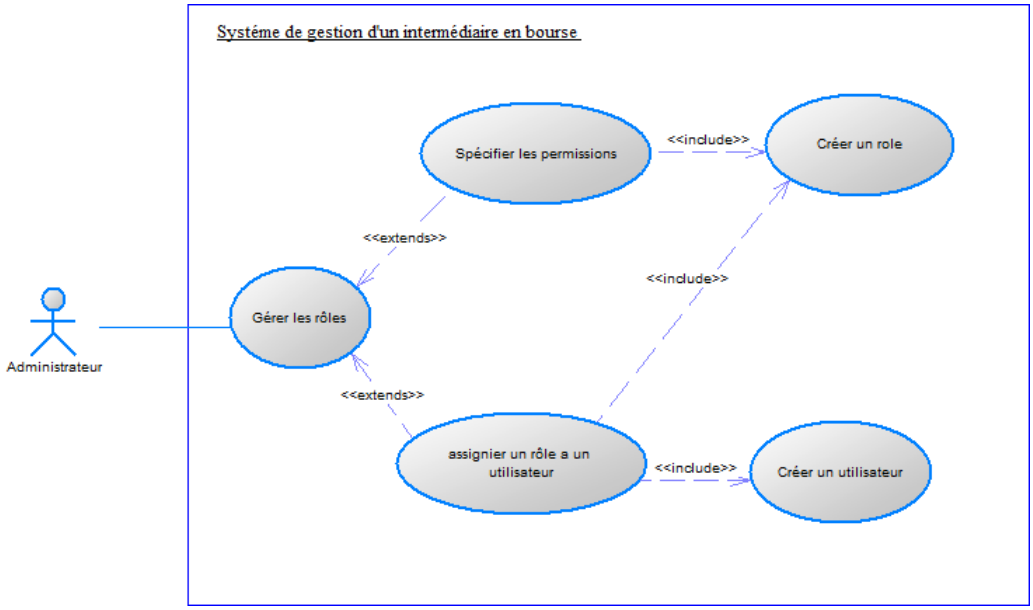



FIGURE 2.4 – Diagramme de cas d'utilisation « Gérer les rôles »

**Diagramme de cas d'utilisation « gérer les ordres et l'exucution manuelle »**

L'exécution manuelle permet de mettre face à face un contrat et un ou plusieurs ordres préalablement saisis. Suite à l'exécution, le portefeuille client ainsi que son solde sont mises à jour après comptabilisation des différents taxes et commissions nécessaires. La figure 2.5 définit le diagramme de cas d'utilisation gérer les ordres et l'exécution manuelle.

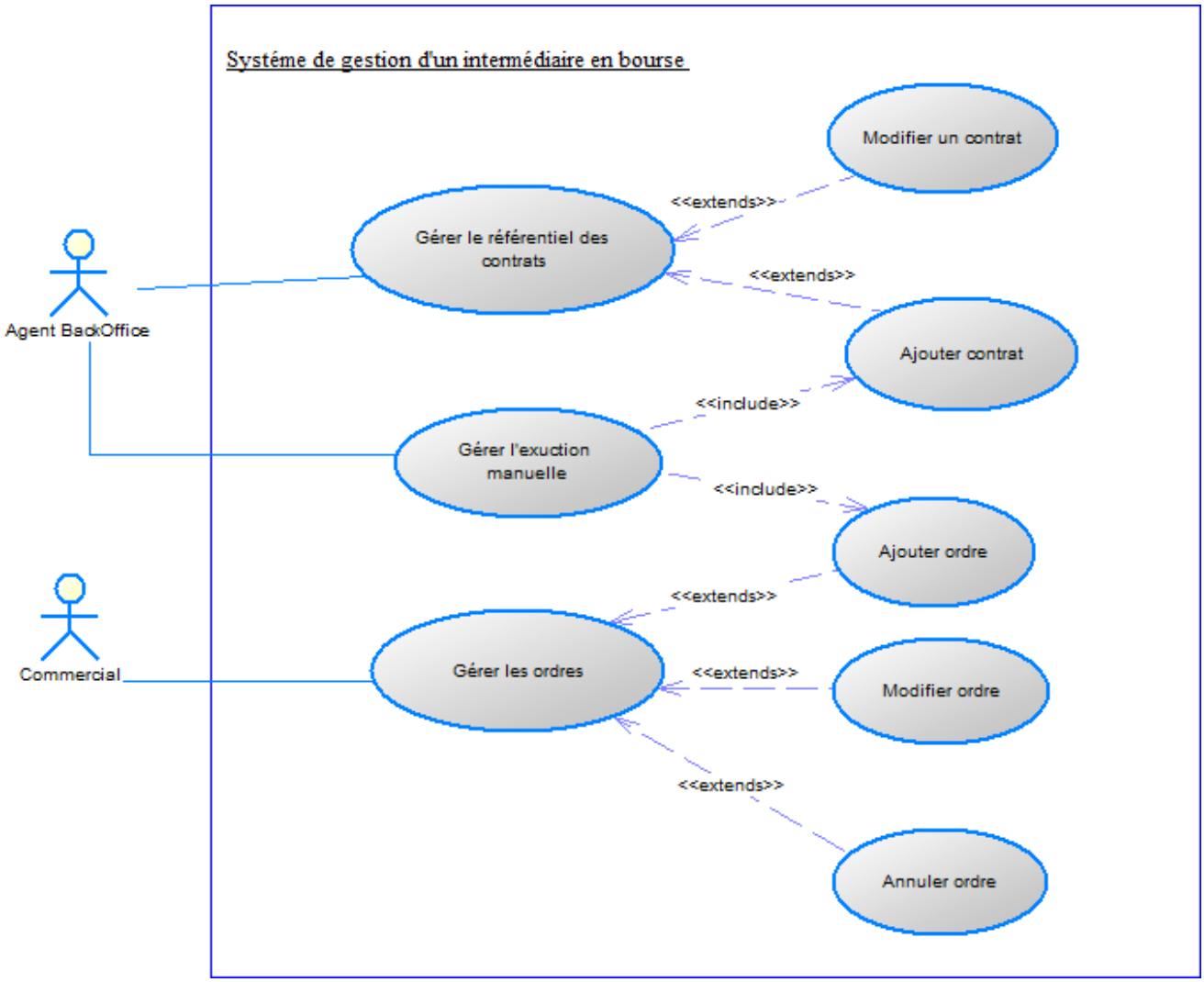



FIGURE 2.5 – Diagramme de cas d'utilisation « gérer les ordres et l'éxecution manuelle »

### 2.2.4 Description détaillée des cas d'utilisations

Dans cette phase, nous allons recenser toutes les interactions avec le système dans le but de décrire la dynamique des cas d'utilisation les plus complexes. Pour chaque cas d'utilisation nous allons identifier les pré-conditions, les scénarios nominaux, les enchainements alternatifs, les exceptions et les post-conditions.

#### 2.2.4.1 Etude du cas d'utilisation « Gérer les ordres »

Ce cas d'utilsation peut être décrit comme suit.

| |
|---|
| **Titre** : Gérer les ordres<br>**But** : Permettre à un utilisateur de gérer le carnet d'ordre.<br>**Résumé** : création, modification, annulation d'un ordre d'achat ou de vente à partir des données saisies par l'utilisateur et filtrage du carnet d'ordres.<br>**Acteurs** : Commercial.<br>**Date de création** : 08/03/2013<br>**Version** : 1.0 |

Les enchaînements suivants peuvent être envisagés.

| |
|---|
| **Pré-conditions :**<br>L'utilisateur est connecté au système et il est autorisé à accéder au module gestion d'ordres. |
| **Scénario nominal :**<br>Ce cas d'utilisation commence lorsque l'utilisateur demande la gestion des ordres.<br><br>**Enchainement (a) Ajouter un ordre :** L'utilisateur doit préciser la valeur, la quantité, le numéro du compte client, le type « risque mangement » , le sens et la validité de l'ordre. S'il s'agit d'un ordre de type limité, « stop loss », ou « stop limit » l'utilisateur doit préciser le cours limite.<br>L'utilisateur peut introduire le type de règlement et une éventuelle observation.<br>Dans le cas où le client va payer par chèque, l'utilisateur doit saisir le numéro de chèque. Si l'utilisateur n'est pas autorisé à traiter le compte en question ou le compte n'existe pas l' [Exception1 : CompteClientInvalide] est exécutée.<br><br>**Enchainement (b) Modifier un ordre :** L'utilisateur décide de modifier un ordre qui existe déjà dans son carnet. Il ne peut pas changer le sens de l'ordre ni la valeur. Si l'utilisateur modifie le type de l'ordre, il doit, éventuellement, préciser les prix. S'il s'agit d'un ordre totalement exécuté, ou annulé alors le système empêche la modification de cet ordre.<br><br>**Enchainement (c) Annuler des ordres :** L'utilisateur doit selectionner la liste d'ordres à annuler.<br><br>**Enchainement(d) Imprimer un ordre :** L'utilisateur doit selectionner l'ordre à imprimer. |

**Enchainement(e) filtrer le carnet d'ordres :**

L'utilisateur doit préciser les critères de filtrage.

**Scénario alternatifs :**

**Enchainement (f)** L'utilisateur décide d'ajouter un ordre d'achat et le prix de l'ordre dépasse la liquidité du client alors [Exception2 : DépassementLiquidité] est exécutée.

**Enchainement (g)** L'utilisateur décide d'ajouter un ordre de vente et la quantité de l'ordre dépasse le stock du client pour la valeur en question alors l'[Exception3 : DépassementDeStock] est exécutée.

**Enchainement (h)** L'utilisateur décide d'ajouter un ordre et la quantité dévoilée de l'ordre dépasse la quantité réelle ou elle est inferieure à 10 alors il faut exécuter [Exception4 : QuantitéeDévoiléeInvalide].

**Exceptions :**

**[Exception1 : CompteClientInvalide]** Un message s'affiche indiquant que l'utilisateur doit saisir un numéro de compte valide et sur lequel il est autorisé de passer des ordres.

**[Exception2 : DépassementLiquidité]** Un message s'affiche à l'utilisateur indiquant la liquidité actuelle du client et la somme manquante et demander à l'utilisateur de ne pas dépasser la liquidité disponible.

**[Exception3 : DépassementDeStock]** Un message s'affiche à l'utilisateur indiquant le Stock actuelle du client et la valeur de dépassement et demander à l'utilisateur de ne pas dépasser le stock disponible.

**[Exception4 : QuantitéeDévoiléeInvalide]** Un message s'affiche à l'utilisateur indiquant que la quantité dévoilée de l'ordre dépasse la quantité réelle ou elle est inférieure à dix

**Post-condition**

**Enchainement(a)** l'ordre est ajouté et imprimé sur l'imprimante locale et un message de confirmation s'affiche.

**Enchainement(b)** L'ordre est modifié.

**Enchainement(c)** Les ordres sélectionnés sont annulés et supprimés du carnet.

**Enchainement(d)** L'ordre est imprimé sur l'imprimante locale.

**Enchainement(d)** Le carnet d'ordres est filtré selon les critères saisies.

#### 2.2.4.2 Etude du cas d'utilisation «Gérer la caisse »

Ce cas d'utilsation peut être décrit comme suit.

**Titre** : Gérer la caisse
**But** : Permettre à un agent de back office la consultation, la recherche et l'ajout des liquidités comme les versements et les retraits de différents types (espèces, chèques, virement , etc.).
**Résumé** : Consultation, recherche, ajout et import de liquidités.
**Acteurs** : Agent de back office.
**Date de création** : 09/03/2013

Les enchaînements relatifs à ce cas sont décrits comme suit.

**Pré-conditions :**
L'utilisateur est connecté au système et il est autorisé à accéder au module gestion de caisse.

**Scénario nominal :**
Ce cas d'utilisation commence lorsque l'utilisateur demande la gestion de caisse.

**Enchainement (a) Ajouter liquidité :** L'utilisateur doit préciser le numéro de client bénéficiaire, le montant, le sens(CREDIT, DEBIT), le type et une éventuelle observation. Dans le cas où le client va créditer par chèque, l'utilisateur doit introduire le numéro du CIN et le numéro du dépositaire. Si le type est Dividende ou Droit, l'utilisateur doit sélectionner la valeur du portefeuille client sur laquelle il va récupérer le dividende ou les droits. Si le type de liquidité est Virement interne l'utilisateur doit introduire le numéro du compte client destinataire. Si l'utilisateur n'est pas autorisé à traiter le compte en question ou le compte n'existe pas l' [Exception1 : CompteClientInvalide] est exécutée. Dans le cas ou le client a déjà passé un ordre de payement l'utilisateur n'a qu'à sélectionner l'ordre de payement préalablement saisi par le commercial.

**Enchainement (b) importer liquidités :** L'utilisateur n'a qu'à sélectionner un fichier ayant une forme bien déterminée contenant une liste de liquidités à importer.

**Scénario alternatifs :**

**Enchainement (c)** L'utilisateur décide d'ajouter un virement interne et il n'est pas autorisé à traiter le compte destinataire alors l'[Exception1 : CompteClientInvalide] est exécutée.

**Enchainement (d)** L'utilisateur décide de débiter un compte et il n'est pas autorisé a effectuer l'opération débit alors l'[Exception2 : DebitImpossible] est exécutée.

**Enchainement (e)** L'utilisateur décide d'importer des liquidités et le fichier d'import n'est pas compatible alors l'[Exception3 : FichierInCompatible] est exécutée.

**Exceptions :**

**[Exception1 : CompteClientInvalide]** Un message s'affiche indiquant que l'utilisateur doit saisir un numéro de compte valide et sur le quel il est autorisé d'ajouter des liquidités.

**[Exception2 : DebitImpossible]** Un message s'affiche à l'utilisateur indiquant qu'il n'est pas autorisé à débiter de l'argent.

**[Exception3 : FichierInCompatible]** Un message s'affiche indiquant que l'utilisateur doit introduire un fichier sous un format compréhensible par le système.

**Post-condition**

**Enchainement(a)** La Liquidité est ajoutée, le solde client est mis à jour, un reçu de paiement est imprimé sur l'imprimante locale et un message de confirmation est affiché.

**Enchainement(b)** La liste des liquidités est importée depuits le fichier et les soldes des clients sont mis à jour.

### 2.2.4.3 Cas d'utilisation « Gérer l'éxecution manuelle »

Afin de mieux comprendre ce cas d'utilisation, nous avons choisi de représenter ces enchaînements avec le diagramme d'activité de la figure 2.6.

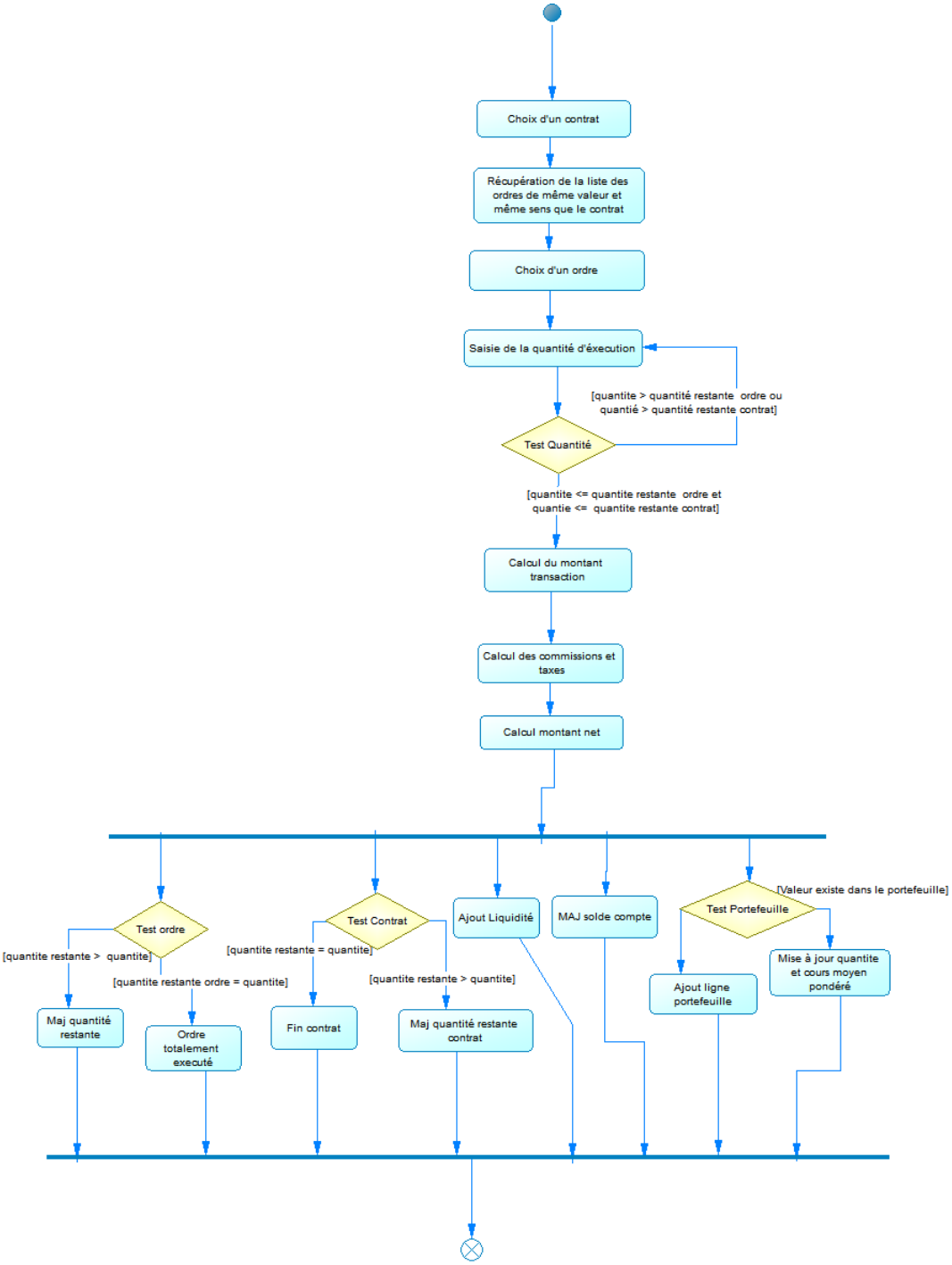



FIGURE 2.6 – Diagramme d'activités éxecution manuelle

### 2.2.5 Besoins non fonctionnels

Dans le but de mieux évaluer les contraintes techniques, nous pouvons ajouter, par ailleurs, les contraintes non fonctionnelles qui sont résumées dans le tableau 2.2.

| Contrainte | Descriptif |
|---|---|
| Temps de réponse | Le système doit répondre en temps réel. |
| Concurrence | Le système doit pouvoir gérer les appels concurrents. |
| Disponibilité | Le système doit être disponible à tous moments. |
| Confidentialité | Les utilisateurs sont authentifiés en fonction de leur login/mot de passe et de leur profil. |

TABLE 2.2 – Contraintes non fonctionnelles

### 2.2.6 Identification des classes candidates

Cette phase permet d'introduire la modélisation orientée objet en mettant à jour les principales abstractions du système sous forme d'objets et de classes. En formalisant les concepts métiers, nous avons pu identifier les classes candidates suivantes : Valeurs, Client, Compte, Portefeuille, Contrat, Intermediaire, Ordre, Execution, OrderDePayement et Liquidités.

## 2.3 Capture des besoins techniques

La capture des besoins techniques est une activité de la branche droite du cycle en Y, comme illustré dans la figure 2.7 ; elle est primordiale pour la phase de conception architecturale. Dans cette partie, nous allons étudier le système de point de vue technique à travers une spécification de l'architecture et une présentation des outils logiciels et matériels sélectionnés.

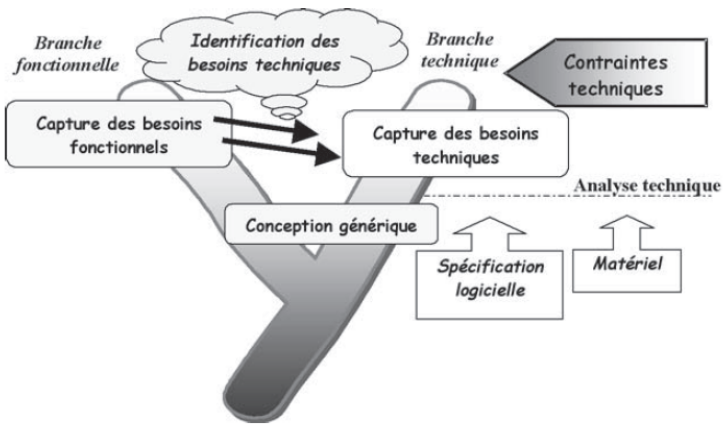



FIGURE 2.7 – Capture des besoins techniques

### 2.3.1 Etude architecturale

Dans cette phase nous allons présenter les différentes architectures potentiellement utilisables dans le cadre de notre projet tout en justifiant le choix architectural adopté.

#### 2.3.1.1 L'architectures N-tiers

L'architecture n-tiers a été créée pour pallier aux limitations des anciennes architectures comme l'architecture 1- tiers, client-serveur et trois tiers et pour concevoir des applications puissantes et simples à maintenir. Ce type d'architecture permet de distribuer plus librement la logique applicative, ce qui facilite la répartition des charges entre tous les niveaux. Cette évolution des architectures trois tiers met en oeuvre une approche objet pour offrir une plus grande souplesse d'implémentation et faciliter la réutilisation des développements. Avec les applications n-tiers, on dispose enfin d'une vision cohérente du système d'information. Tous les services sont représentés sous la forme d'objets interchangeables qu'il est possible d'implanter librement en fonction des besoins. Le potentiel de ce type d'application est très important et permettra enfin l'utilisation des objets métiers réutilisables.

#### 2.3.1.2 L' architecture orientée service « SOA »

L'architecture orientée service noté en littérature SOA [1] constitue un style d'architecture basée sur le principe de séparation de l'activité métier en une série de services. Ces services peuvent être assemblés et liés entre eux selon le principe de couplage lâche pour exécuter l'application désirée. Ces services sont définis à un niveau supérieur de la traditionnelle approche composants.

#### 2.3.1.3 Compatibilité entre SOA et DDD

Les deux concepts SOA et DDD peuvent paraître à première vue totalement différents : Le DDD supporte une architecture N-tires pilotée par le domaine tandis que le SOA est une architecture pilotée par les services. Cepandant, en approfondissant notre recherche, on se rend compte que ces deux paradigmes sont compatibles et peuvent être complémentaires pour construire un nouveau concept celui de DDDD « Distributed Domain Driven Design ». En effet, une couche fine supplémentaire de service distribuée est ajoutée à l'architecture proposée par le Domain Driven Design, chaque web service s'appui donc sur une architecture multicouche faiblement couplée qui suit le concept du Domain Driven Design. La figure 2.8 explique l'architecture DDDD que nous avons choisi d'adôpter pour réaliser notre projet.

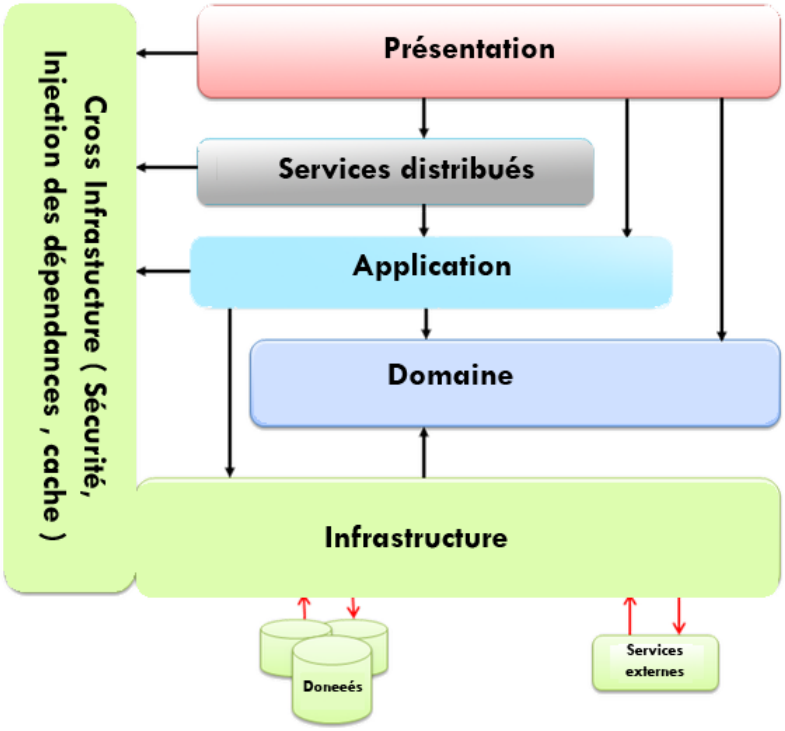



FIGURE 2.8 – Architecture DDDD

### 2.3.2 Etude technologique

Dans cette section, nous allons présenter l'ensemble des technologies potentiellement utilisables dans le cadre de notre projet tout en justifiant notre choix.

#### 2.3.2.1 .NET Framework

C'est un ensemble de produits et de technologies informatiques de Microsoft pour rendre des applications facilement portables sur Internet. Il offre une vaste bibliothèque permettant aux développeurs d'éviter plusieurs problèmes de programmation et par suite rendre le développement plus facile. Ainsi Microsoft .Net fournit un serveur web local permettant de gérer des services et évitant d'externaliser des données privées sur un service web de stockage ou un hébergement web tiers et une machine virtuelle pour l'exécution basée sur le CLR (Common Langage Runtime) il s'agit de l'implémentation par Microsoft du standard Common Langage Infrastructure (CLI) qui définit l'environnement d'exécution des codes de programmes basé sur le CIL (Common Interface Langage).

#### 2.3.2.2 Les Applications Internet Riches (RIA)

L'une des évolutions majeures que le Web a connu est sans aucun doute est l'apparition des applications internet riche dites RIA(Rich Internet Application).

Une application (RIA), est une application web qui offre des caractéristiques similaires aux logiciels traditionnels installés sur un ordinateur.

On peut cité d'autre avantage de ce genre d'application :

- Aucune installation est nécessaire à part l'environnement (navigateur Web et un éventuelle plugin) ce qui a pour conséquences de diminuer le cout de maintenance et déploiement.

- Les données sont sauvegardées sur un serveur sécurisé.

- La facilité d'accès a l'environnement (application et données) depuis n'importe quel ordinateur.

Il existe différents types de « frameworks » pour réaliser une application RIA et chacun a ses avantages et ses inconvénients :

- HTML5 : C'est une collaboration entre le World Wide Web Consortium (W3C) et le Web Hypertext Application Technology Working Group (WHATWG). L'un des principaux avantages de HTML5 réside au fait qui'il réduit le besoin des plugins externes (comme Flash), offre des nouvelles fonctionnalités fondées sur le langage HTML, CSS, DOM et JavaScript et propose une meilleure gestion des erreurs [11].
- Silverlight : C'est une technologie de plug-in pour navigateur développée par Microsoft, Cette technologie a pris, au cours des dernières années, une place importante dans le développement d'applications RIA(Rich Internet Application). Cette technologie, initialement perçue comme une plateforme pour intégrer des multimédia aux applications web, mais elle a rapidement gagné du terrain dans des domaines où elle n'était pas attendue comme les RDA (Rich Desktop Application) et le développement des applications métiers fortement orientées données[9].
- Choix adopté : L'interface Homme-Machine a une importance capitale dans les nouvelles applications. Nous avons choisi d'utiliser Silverlight puisqu'il offre des performances nettement supérieures à HTML5 et vu qu'il est utilisé dans d'autres produits de «CybEx solutions» ce qui facilite l'intégration dans le système déjà établi. De plus, nous allons l'enrichir avec quelques bibliothèques essentiellement celle de Telirik, car elles offrent un nombre de composants plus évolués que les composants natifs.

#### 2.3.2.3 Windows Communication Foundation (WCF)

Windows Communication Foundation (WCF) [10] est un Framework qui facilite le développement des applications orientées services. Il représente l'un des composant de base de .NET Framework3.0 avec ( WPF,WF et CardSpace). Un service WCF est une entité logicielle définit par les trois lettres, A, B, C.

- A : acronyme d'Address : où se localise le service ?
- B : acronyme de Binding : comment peux-je communiquer avec le service ?

- C : acronyme de Contract : que peut le service m'offrir ?

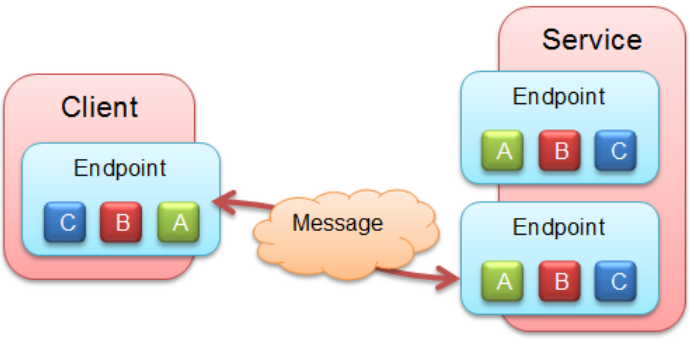



FIGURE 2.9 – Architecture logicielle du WCF

**2.3.2.4 Object relational mapping(ORM)** Un mapping objet-relationnel (en anglais object-relational mapping ou ORM) est une technique de programmation informatique qui crée l'illusion d'une base de données orientée objet à partir d'une base de données relationnelles en définissant des correspondances entre cette base de données et les objets du langage utilisé. On pourrait le désigner par « correspondance entre monde objet et monde relationnel.

Entity framework et NHibernate sont les deux ORM les plus utilisées qui peuvent être intégrés à un environnement .NET. Après une comparaison des différentes caractéristiques et des performances de chacun nous ce rond compte que les deux framework ont à peu près les mêmes capacités. Donc nous avons décidé de choisir Entity framework, car il est supporté par Microsoft.

## 2.4 Conclusion

A travers ce chapitre, nous avons pu compléter le recueil des besoins fontionnels et techniques et préparer l'analyse orientée objet qui sera traitée dans de chapitre suivant , et ce en identifiant les classes candidates du modèle statique d'analyse.

Chapitre 3

# Analyse

## 3.1 Introduction

Suite à la phase du capture des besoins, nous avons pu obtenir un découpage fonctionnel exprimé à travers les cas d'utilisation organisés dans le modèle de spécification fonctionnelle. Dans la phase d'analyse nous allons changer radicalement l'organisation du modèle. À cet effet, nous allons passer d'une structuration fonctionnelle via les cas d'utilisation à une structuration orientée objet via les classes et les catégories, et ce comme le montre la figure 3.1.

Dans ce chapitre, nous allons présenter le développement du modèle statique et dynamique du système en commençant par le découpage en catégories.

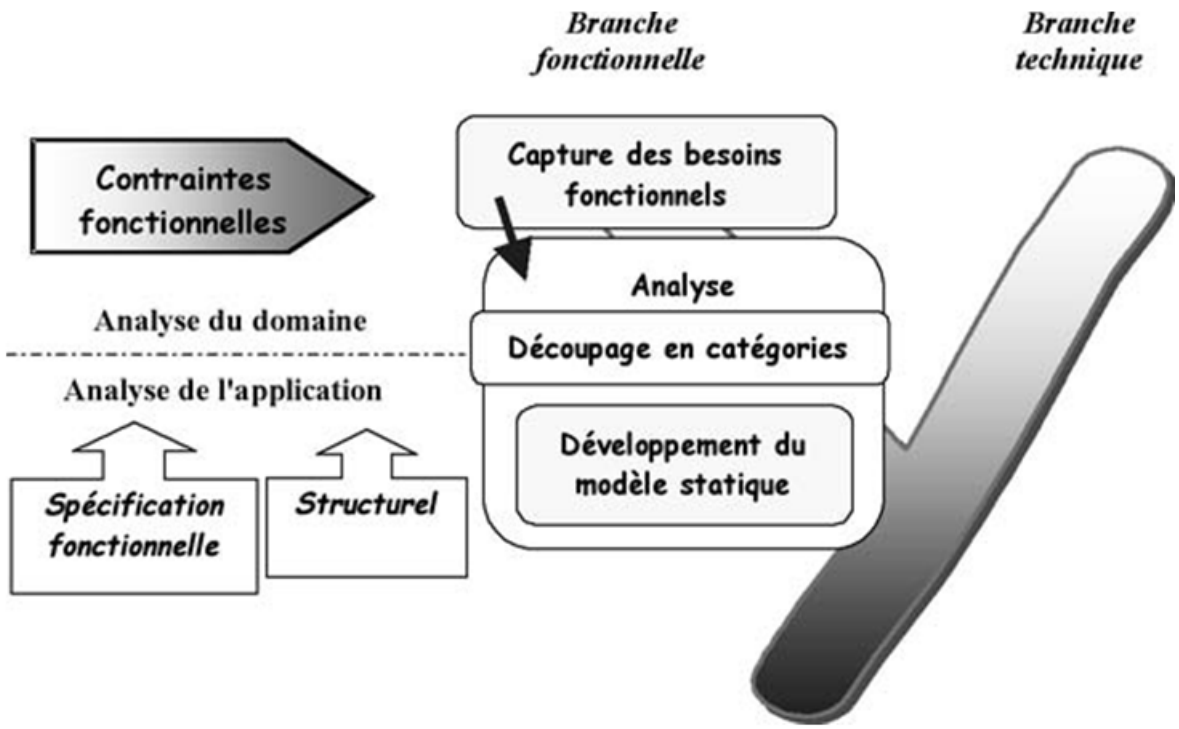



FIGURE 3.1 – L'analyse

## 3.2 Découpage en catégories

Le découpage en catégories constitue la première activité de l'étape d'analyse. Il se situe sur la branche gauche du cycle en Y et succède à la capture des besoins fonctionnels. Elle utilise la notion de paquetage pour définir des catégories de classes d'analyse et découper le modèle en blocs logiques les plus indépendants possibles.

Nous avons définit cinq catégories à savoir : Client, Ordre, Administration, Liquidité et Exécution.Ceci est illustré par la figure 3.2.




FIGURE 3.2 – Découpage en catégories

### 3.2.1 Dépendance entre les catégories

Après avoir identifié les différentes catégories ainsi que les classes candidates, nous pouvons, à ce stade, décider des dépendances entre les catégories, comme expliqué dans le diagramme de la figure 3.3

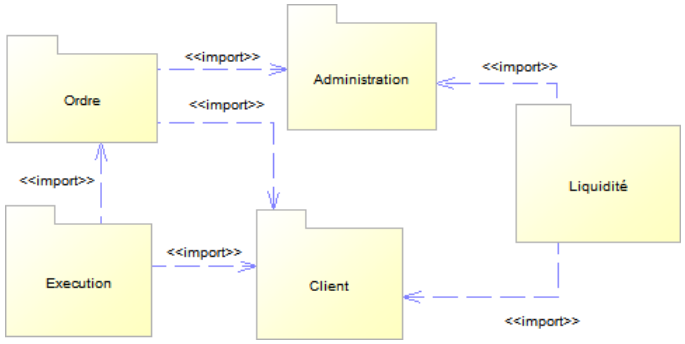



FIGURE 3.3 – Dépendance entre les catégories

## 3.3 Développement du modèle statique

Le modèle statique représente le processus d'amélioration des diagrammes de classes candidates vers des modèles finaux. Cette partie consiste à affiner les classes candidates, affiner

les associations, supprimer les redondances et ajouter les attributs et les méthodes. Dans ce modèle, nous ne conservons que les propriétés des classes que le système doit mémoriser et utiliser, de même les opérations non métiers ou implicites ne seront rajoutées que dans le chapitre « Conception ». Après avoir affiné les classes et les associations et ajouter des attributs et des opérations, nous obtenons les modèles suivants.

### 3.3.1 Modèle structurel de la catégorie Client

Le diagramme de la figure 3.4 explique le modèle préliminaire de la catégorie client.

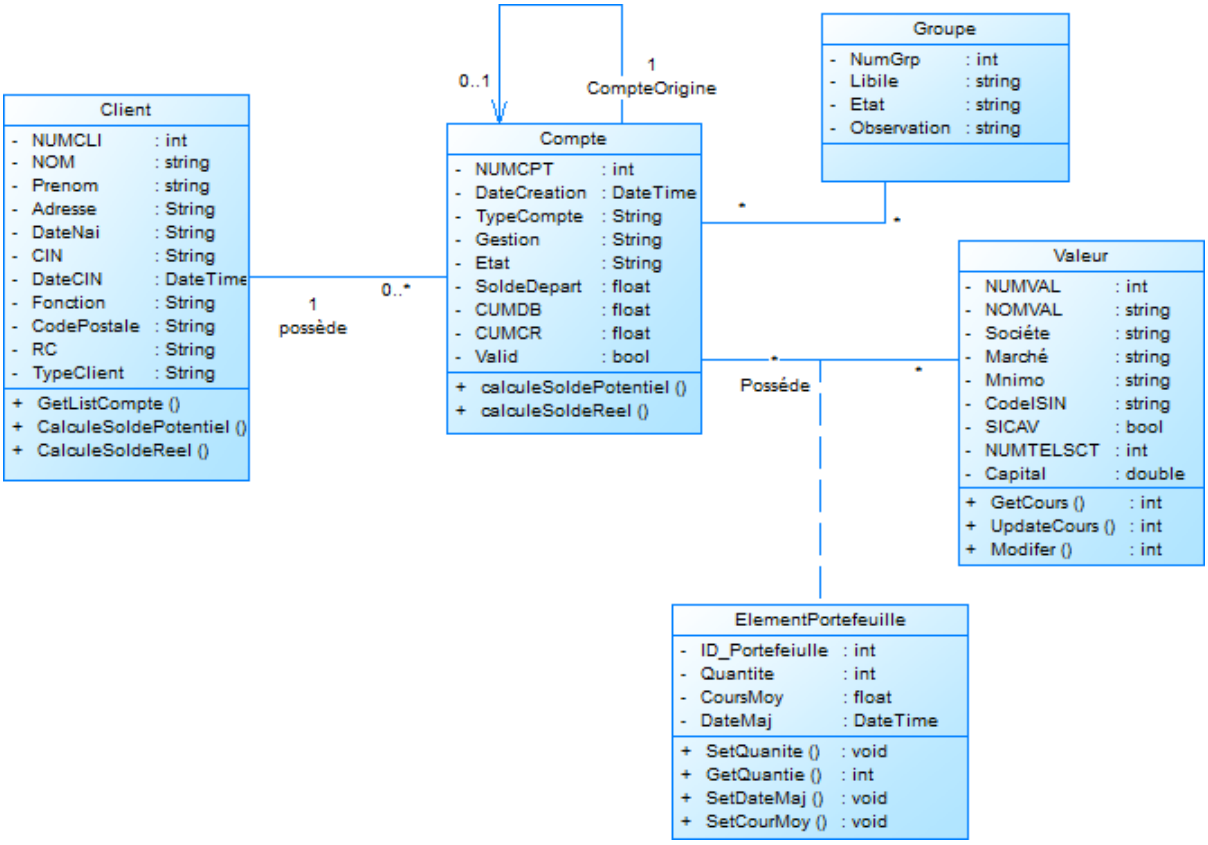



FIGURE 3.4 – Diagramme de classe préliminaire de la catégorie client

- La classe « Client » recense les informations nécessaires à un client de l'intermédiaire. Un client peut posséder un ou plusieurs comptes.
- La classe «Compte» recense les informations nécessaires d'un compte titre. Les comptes peuvent être regroupés dans un ou plusieurs groupes.

Les comptes de type CEA doivent référer sur un compte d'origine de type Compte Ordinaire.

- La classe «Valeur» recense les différentes informations pour les valeurs exécutées sur le marché.
- La classe «ElementPortefeuille» construit la composition du portefeuille d'un compte donné en précisant la quantité qu'il dispose pour chaque valeur.

### 3.3.2 Modèle structurel de la catégorie Ordre

Le diagramme de classe préliminaire de la catégorie ordre est décrit par la figure 3.5.

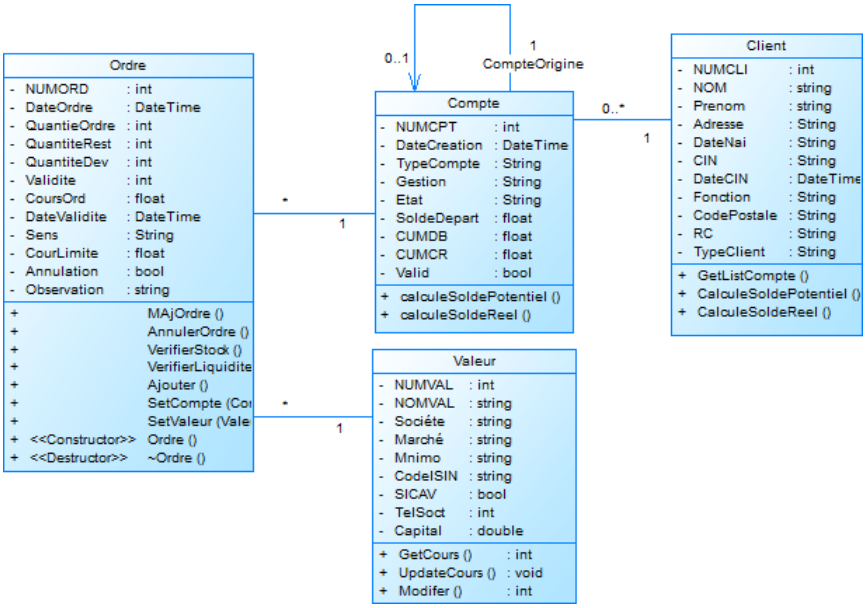



FIGURE 3.5 – Diagramme de classe préliminaire de la catégorie ordre

La classe «Ordre» permet de construire le carnet d'ordre, d'assurer les opérations de création, de modification, d'annulation et d'envoi des ordres. Elle recense toutes les informations nécessaires pour un ordre comme le sens, la validité le type, cours, etc.

### 3.3.3 Modèle structurel de la catégorie Exécution

Le modèle structurel de la catégorie « Exécution » est détaillé par la figure 3.6.

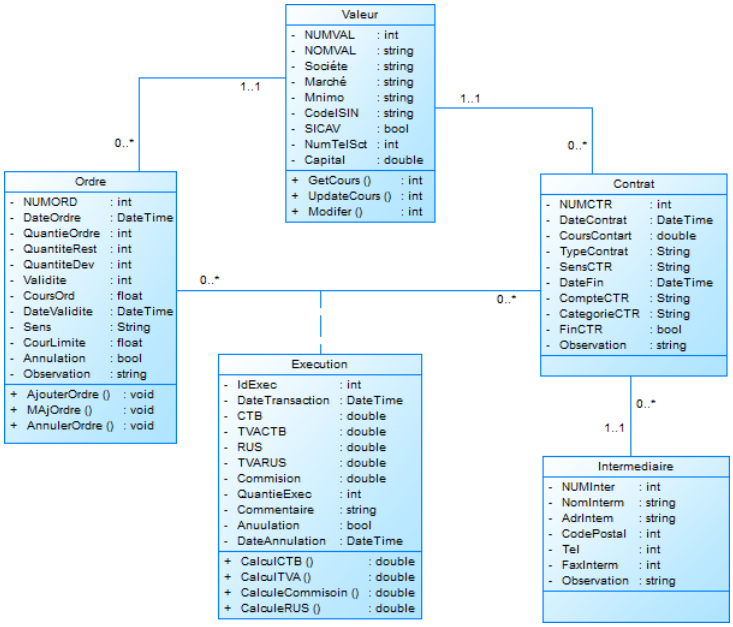



FIGURE 3.6 – Diagramme de classe préliminaire de la catégorie Exécution

- La classe «Execution» enregistre toutes les informations des transactions d'achat ou de vente effectuées sur un contrat. A partir de cette classe, on peut ainsi mettre à jour le portefeuille client ainsi que son solde après avoir calculer les différents commissions et taxes.
- La class «Contrat» enregistre toutes les informations des contrats entre l'intermédiaire utilisateur du système et les autres intermédiaires pour l'achat ou la vente d'une quantité d'une valeur donnée.
- La classe « Intermédiaire» recense toutes les informations nécessaires pour les différents intermédiaires qui exercent sur le marché.

### 3.3.4 Modèle structurel de la catégorie Liquidité

Le modèle structurel de la catégorie « Liquidité » est détaillé par la figure 3.7.

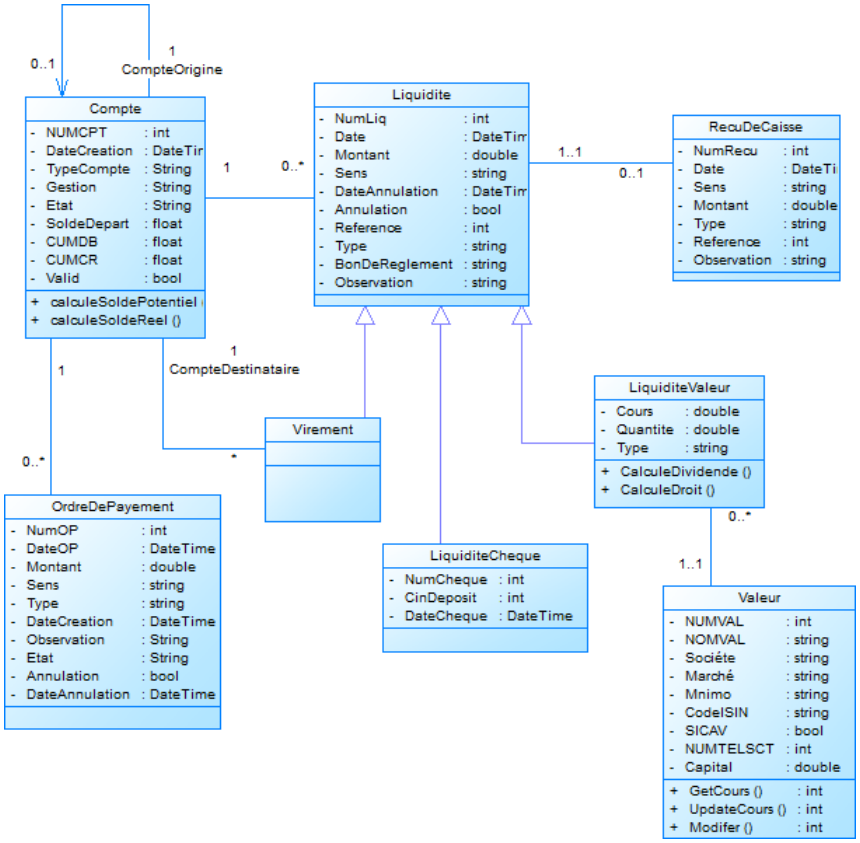



FIGURE 3.7 – Diagramme de classe préliminaire de la catégorie Exécution

- La Classe « Liquidite » encapsule toutes les informations nécessaires pour enregistrer les opérations de versement ou de retrait d'argent de différents types : espèce, virement, frais etc.
- La classe « LiquiditeCheque » hérite de la classe « Liquidite » en ajoutant les informations nécessaires pour les transactions par chèque.
- La classe « LiquideValeur » hérite de classe « Liquidite » en ajoutant les informations nécessaires pour les liquidités sur la valeur mobilière comme les dividendes et les droits.
- La classe « Virement »hérite de la class « Liquidite » tout en précisant le compte destinataire du virement
- La classe «OrdreDePayement» recense toutes les informations indispensables pour enregistrer les ordres de payement passés par les commerciaux pour le bénéfice d'un compte et qui seront par la suite transformés en liquidité.
- La classe « Reçu de caisse » encapsule les informations du reçu de caisse fourni au client.

## 3.4 Développement du modèle dynamique

Le développement du modèle dynamique constitue la troisième activité de l'étape d'analyse. Cette étape va nous permettre d'illustrer l'aspect dynamique du système tout en décrivant des scénarios mettant en jeu un ensemble d'objets échangeant des messages. Ces interactions sont décrites au moyen de deux types de diagrammes : le diagramme de séquence, qui met l'accent sur la chronologie des messages et le diagramme de communication qui souligne les relations structurelles des objets en interaction.

Nous présentons, dans ce qui suit, les modèles dynamiques des cas d'utilisation les plus complexes.

### 3.4.1 Cas d'utilisation : Gérer les ordres

En prenant le cas d'utilisation Gérer les ordres, nous avons définit les scénarios suivants.

#### 3.4.1.1 Scénarios nominaux

- GO_N1 : Ajout d'un ordre
- GO_N2 : Modification d'un ordre
- GO_N3 : Annulation d'un ordre
- GO_N4 : Filtrer le carnet d'ordres

#### 3.4.1.2 Scénarios alternatifs

- GO_A1 : Création d'un ordre avec un numéro de compte client invalide
- GO_A2 : Création d'un ordre avec une quantité dévoilée invalide
- GO_A3 : Création d'un ordre avec un prix invalide
- GO_A4 : Création d'un ordre en dépassant la liquidité du client
- GO_A5 : Création d'un ordre en dépassant le stock disponible du client.

#### 3.4.1.3 Formalisation des scénarios

Parmi tous les scénarios possibles pour le cas d'utilisation : « Gérer les ordres » nous avons choisi de formaliser GO_N1 et GO_A5.

Prenons le scénario GO_N1 : «Ajout d'un ordre». Pour décrire ce scénario, nous allons faire intervenir les lignes de vie suivantes :

- Un acteur commercial.

- Un objet « nouvelOrdre : Ordre » représentant l'ordre crée au cours du scénario.
- Un objet « Client :client» représentant le client bénéficiaire de l'ordre.
- Un objet « Compte » représentant le compte du client bénéficiaire de l'ordre.
- Un multi-objet « ListeValeurs » représentant les valeurs.
- Un objet « lignePortefeuille :ElementPortefeuille » représente la ligne du portefeuille correspondante à la valeur sélectionnée.

Le diagramme de séquence formalisant le scénario GO_N1 est présenté par la figure 3.8.

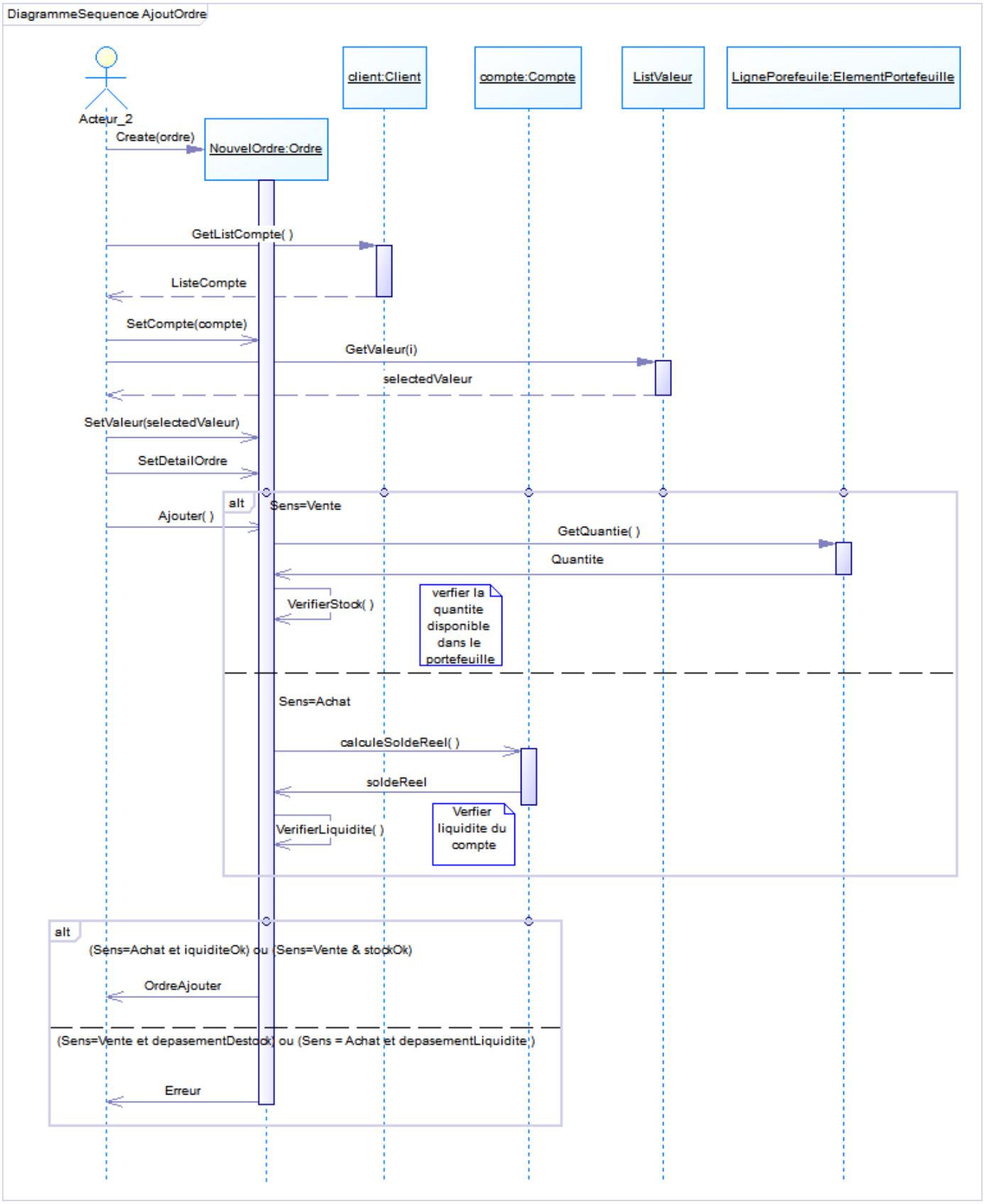



FIGURE 3.8 – Diagrammede séquence ajout d'un ordre

L'opération d'ajout d'un ordre passe par des étapes précises comme suit : Le commercial choisit le client bénéficiaire et récupère la liste des comptes associés, de laquelle il choisit un compte. Il enchaîne l'opération par la sélection d'une valeur de la liste des valeurs disponibles.

Pour clôturer l'opération de saisie, il est appelé à passer les informations propres à cet ordre comme le type, le sens, la validité etc.

Selon le sens de l'ordre, une opération de vérification sera lancée et un message de validation sera communiqué au commercial pour finaliser cette opération.

Nous allons compléter ce diagramme de séquence du scénario en décrivant l'opération « VerifierStock » via le diagramme de communication de la figure 3.9.

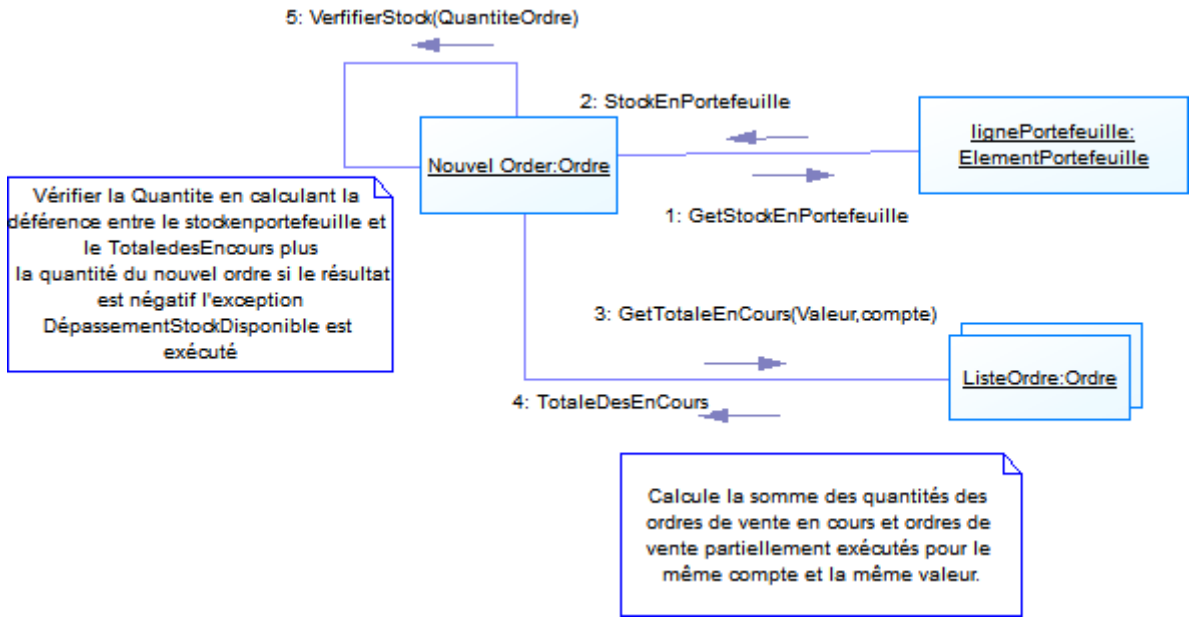



FIGURE 3.9 – Diagramme de communication VerifierStock

### 3.4.2 Cas d'utilisation : Gérer l'exécution manuelle

En prenant le cas d'utilisation « Gérer l'exécution manuelle », nous avons défini les scénarios suivants :

#### 3.4.2.1 Scénarios nominaux

- GEM_N1 : exécution nominale.

#### 3.4.2.2 Scénarios alternatifs

- GEM _A1 : Exécution dépassant la liquidité du compte.
- GEM _A2 : Exécution dépassant la quantité restante dans le contrat.
- GEM _A3 : Exécution dépassant la quantité restante dans l'ordre.
- GEM _A4 : Exécution dépassant le stock disponible dans le portefeuille.

#### 3.4.2.3 Formalisation des scénarios

Parmi tous les scénarios possibles pour le cas d'utilisation : « Gérer l'exécution manuelle» nous avons choisi de formaliser le scénario GEM_N1 Pour décrire ce scénario, nous allons faire intervenir les lignes de vie suivantes :

Un acteur Agent de back office.

Un objet «execution : Execution » représentant l'exécution créée au cours du scénario.

Un objet « ordre :Ordre» représentant l'ordre à exécuter.

Un objet « compte :Compte » représentant le compte bénéficiaire de l'exécution.

Un multi-objet « ListeContrat » représentant la liste des contrats, non encore achevés ou annulés, visibles pour l'agent de back office.

Un multi-objet « ListeOrdre» représentant la liste des ordres de même valeur et même sens que le contrat sélectionné, visible pour l'agent de back office.

Le diagramme de séquence formalisant le scénario GO_N1 est présenté à la figure suivante :

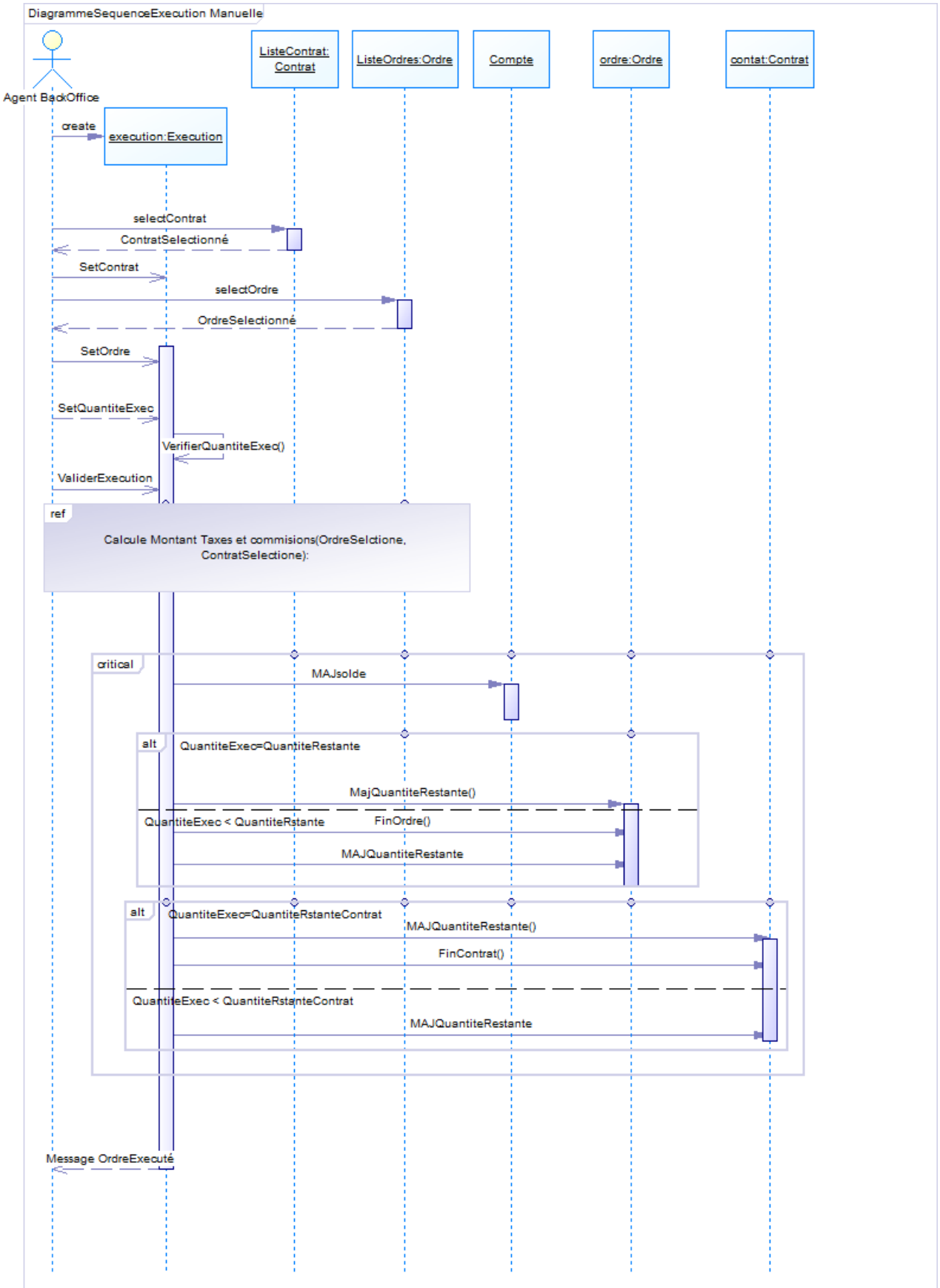



FIGURE 3.10 – Diagramme de séquence exécution manuelle

L'opération d'exécution passe par les étapes suivantes :

- L'agent de back office sélectionne un contrat parmi la liste des contrats sur lesquelles il est autorisé à réaliser des opérations d'exécutions.
- Il choisit un ordre parmi la liste associée au contrat (de même valeur, sens et type) et il ne lui reste plus qu'à préciser une quantité d'exécution valide puis valider l'opération.
- Une fois l'agent de back office a confirmé son opération, un processus de calcul des taxes et des commissions est lancé avant d'entrer dans une zone critique qui prend la responsabilité de mettre à jour le solde du client et les quantités restantes de l'ordre et du contrat.
- Si la quantité restante dans l'ordre ou dans le contrat correspond à la quantité d'exécution, ce dernier est marqué comme totalement exécuté.

## 3.5 Conclusion

Dans ce chapitre, les modèles statiques et dynamiques de notre système ont été mises au point. Ceci a pour conséquence de nous aider à entamer la phase de conception qui sera l'objectif du chapitre suivant.

Chapitre 4

# Conception

## 4.1 Introduction

Après avoir compléter la phase d'analye, nous pouvons désormais, aborder l'étude conceptuelle. Au cours de ce chapitre, nous allons passer par trois phases principales du processus 2TUP, à savoir, la conception générique, la conception préliminaire et la conception détaillée comme le montre la figure 4.1. Pour chaque phase, nous avons choisi de détailler les étapes les plus importantes.

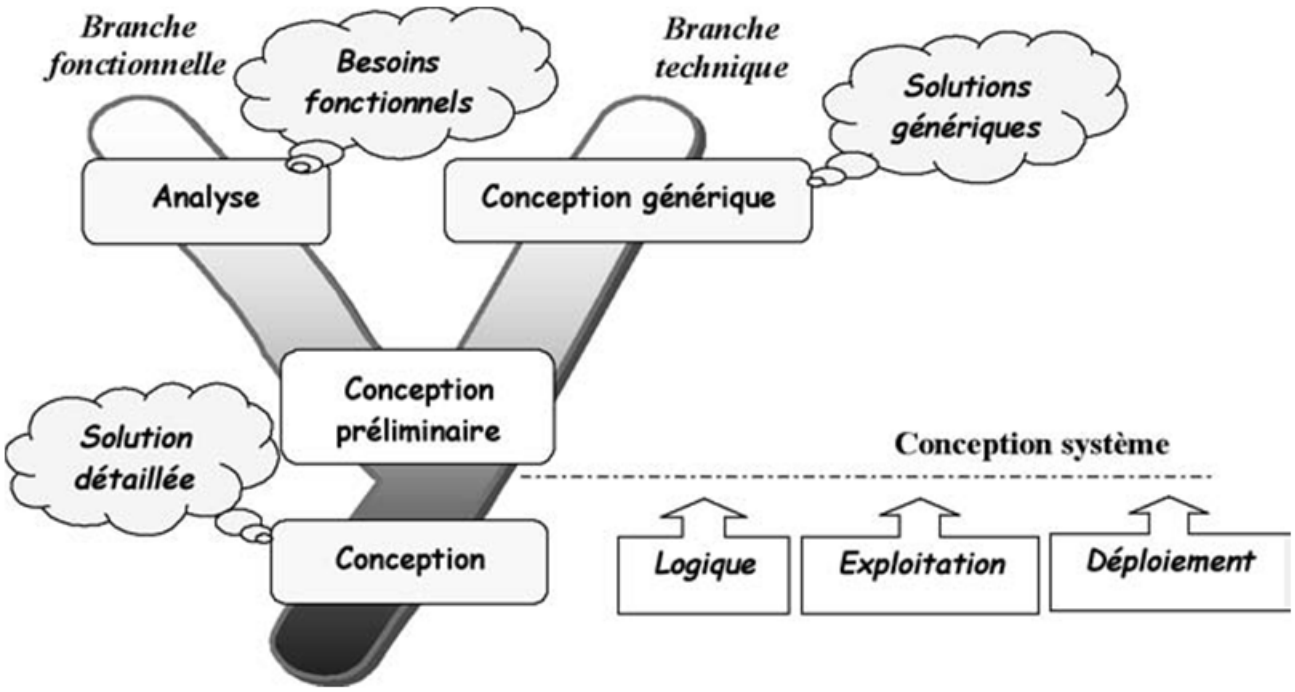



FIGURE 4.1 – Conception

## 4.2 Conception générique

La conception générique explique comment, à partir de l'analyse des besoins techniques, il est possible de commencer à concevoir le système indépendamment de son contenu fonctionnel. Cette partie du manuscrit est dédiée pour la présentation des « Framework » techniques utilisés ainsi que les « Design Patterns » mis en œuvre.

### 4.2.1 Framework techniques

En tenant compte de l'architecture du système établie précédemment, des choix techniques s'imposent. Le tableau 4.1 donne un descriptif des besoins et des choix de « Framework » techniques appliqués.

| Besoin technique | Framework technique |
|---|---|
| .NET Framework | Microsoft .NET Framework 4.0 |
| ORM | Entity Framework V4.0 |
| Framework de communication | WCF |

TABLE 4.1 – Choix techniques

### 4.2.2 Les patrons de conception

L'utilisation des patrons de conception « Design Patterns » offre de nombreux avantages ; ils permettent de répondre à un problème de conception grâce à une solution éprouvée et validée par des experts. Ainsi, on gagne en rapidité de conception et en qualité. De plus, les « Design Patterns » permettent de mettre en avant les bonnes pratiques de conception. Nous précisons dans ce qui suit les « Design Patterns » utilisés dans le cadre de ce projet tout en explicitant leurs apports et avantages.

#### 4.2.2.1 Le Design Pattern « Specification »

Le design pattern « Specification » est utilisé pour gérer les règles métiers d'accès aux données. Il permet la centralisation, l'encapsulation et la composition de ces règles, ce qui facilite leurs réutilisations et communications. Une fois spécifiées, ces règles seront par la suite utilisées pour récupérer les données à partir des entrepôts « repository ».

Le diagrame de classe du Design Pattern « Specification » est représenté par la figure 4.2.

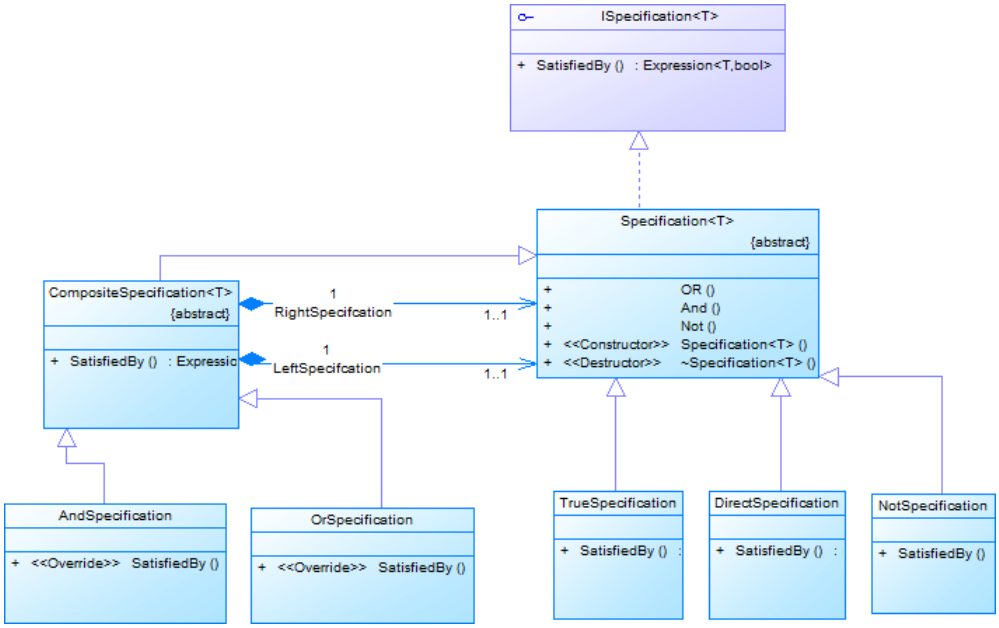



FIGURE 4.2 – Design Pattern « Specification »

#### 4.2.2.2 Le Design Pattern Entrepôt de donnée « Repository »

Le Design Pattern Entrepôt de donnée est utilisé pour assurer la communication avec les sources de données. Contrairement au design pattern DAO « Data Acces Object» qui représente une abstraction de la méthode de persistance, l'entrepôt de donnée représente une abstraction d'une collection de données sous forme d'objets de domaine et assure les fonctionnalités de récupération et de recherche. Les méthodes d'ajout et de modification doivent passées à traves les unités de travail «Unit Of Work».

#### 4.2.2.3 Le Design Pattern «UnitOfWork »

Le Design Pattern « UnitOfWork » est utilisé pour assurer la conservation des objets affectés par une transaction et coordonner l'écriture, la modifications et la résolution des problèmes de simultanéité d'accès aux données.

#### 4.2.2.4 L'inversion de Contrôle « IoC » et l'injection des dépendances

L'inversion de contrôle « IoC » est un patron d'architecture permettant d'éviter les dépendances entre les blocs d'une application. Il existe différentes formes ou représentation d'IoC dont le plus utilisé est l'injection des dépendances. Ce dernier consiste à créer dynamiquement (injecter) les dépendances entre les différentes classes en s'appuyant sur une description (fichier de configuration ou métadonnées) ou de manière programmatique. Ainsi, les dépendances entre composants logiciels ne sont plus exprimées dans le code de manière statique mais déterminées dynamiquement à l'exécution.

#### 4.2.2.5 Le Design Pattern « Data Transfer Object »

Le design pattern « Data Transfer Object » est souvent introduit afin de satisfaire une requête complexe du client, dans une application distribuée, sans émettre un nombre important d'appels vers une interface distante et dans le but d'optimiser la quantité de données transférées sur le réseau. La solution est consiste alors à la création d'un DTO « Data Transfer Object » qui contient toutes les données nécessaires à l'appel distant. Un DTO est un simple conteneur pour un ensemble de données agrégées qui doivent être transférées d'un processus à un autre à travers le réseau. Il ne doit pas contenir de logique métier et son comportement doit être limité à des activités de vérification de la cohérence interne et de validation de base.

#### 4.2.2.6 Le Design Pattern « MVVM »

Le design pattern « Model – View –ViewModel »[3] est spécifique pour l'environement Silverlight et WPF « Windows Presentation Fondation ».Il incorpore de nouveaux éléments par rapport à ses antécédents, à savoir, MVC « Modèle – Vue – Contrôleur » , VP « View-Presenter » et MVP « Model- View-Presenter » et ce, afin d'assurer une meilleure séparation entre la couche métier, l'accès aux données et l'interface utilisateur. La figure 4.3 montre les interactions entre les différents blocs de M-V-VM.

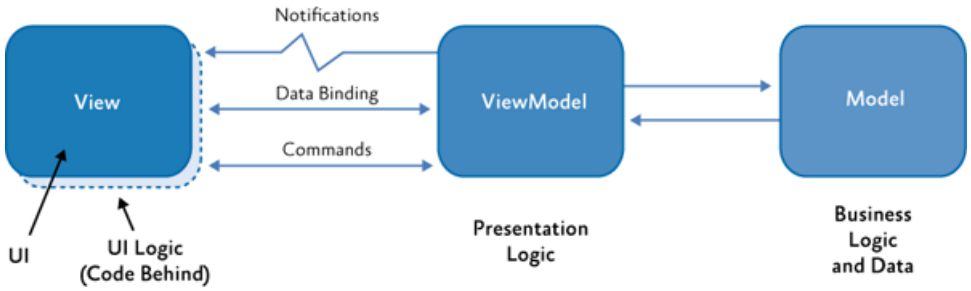



FIGURE 4.3 – Les différents blocs du Design Pattern «MVVM »

La couche « View » permet de gérer l'affichage et l'IHM à travers une description en XAML[4]. Elle peut contenir du code-behind juste pour gérer les animations et la logique propre à l'interface.

La couche « ViewModel » code la logique de présentation. Elle est liée à « View » à traves le « Data-binding »[4] du XAML qui consiste à lier une propriété d'un objet à celle d'un autre d'une façon déclarative.

La couche « Model » englobe toutes les données de l'application ainsi que la logique métier. Il est en communication avec le ViewModel afin de fournir les informations nécessaire.

La figure 4.4 illustre bien la décomposition et le partage des rôles entre les diffèrents composants du modèle MVVM.

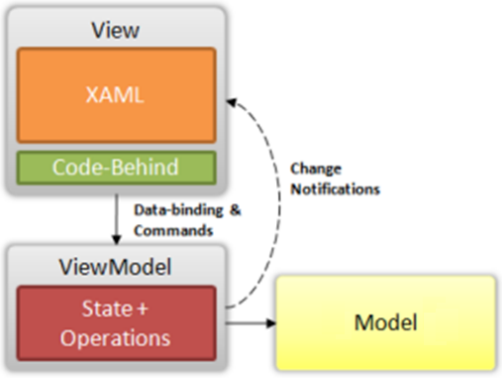



FIGURE 4.4 – Rôles des différents composants du Design Pattern «MVVM»

## 4.3 Conception préliminaire

La conception préliminaire est la phase de rencontre des études fonctionnelles et techniques. Durant cette étape, nous allons concevoir le modèle logique et la structure de la couche présentation du projet.

### 4.3.1 Conception du modèle logique

Le modèle logique permet de représenter l'architecture globale du système. En appliquant les différents patrons de conception listés précédemment, nous avons abouti au modèle logique représenté par la figure 4.5.

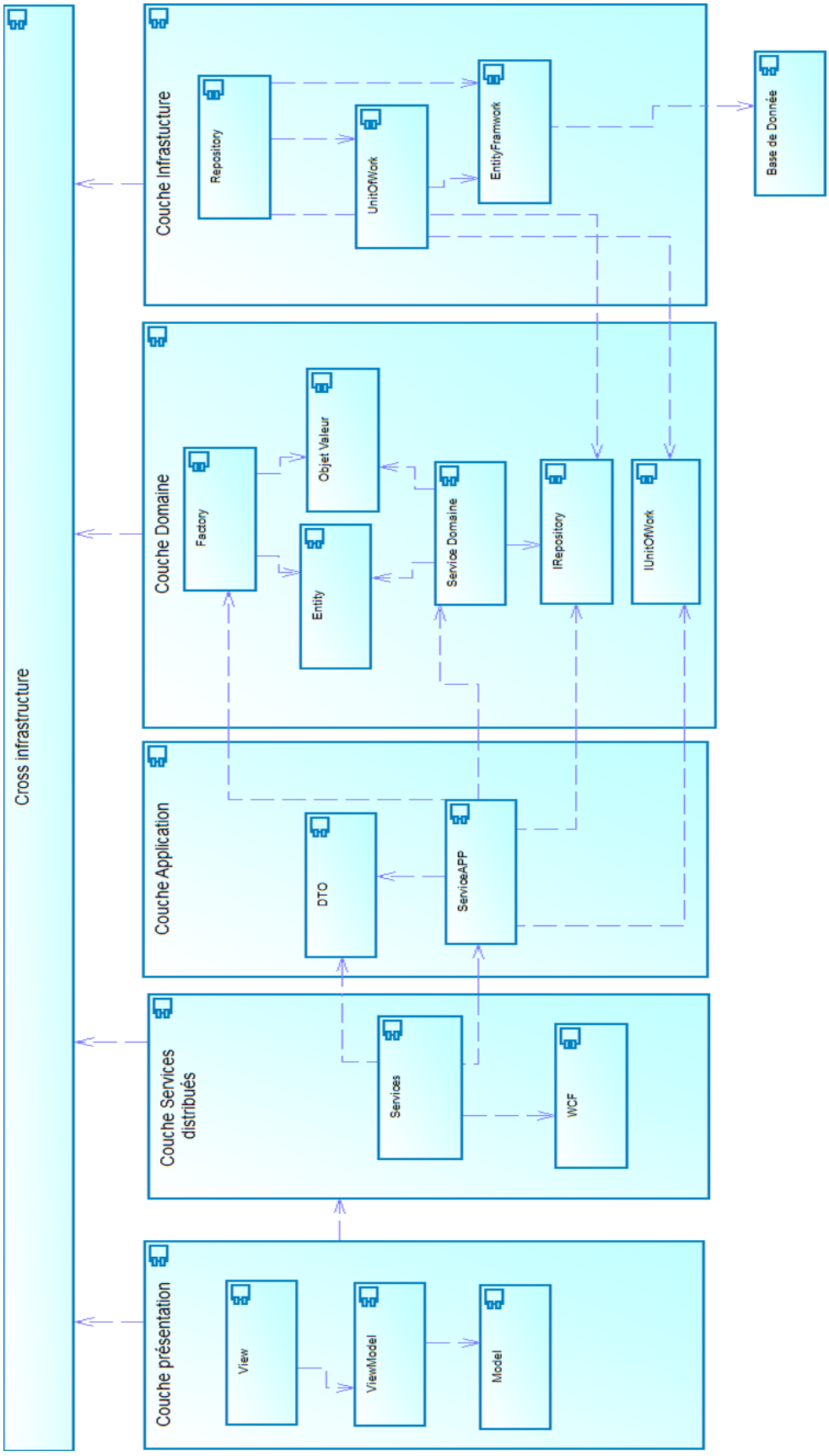



FIGURE 4.5 – Architecture du système de gestion d'un intermédiaire en bourse.

Le système est défini par l'ensemble des composants suivants :

- « View » représente l'ensemble des interfaces graphiques,
- « ViewModel » défini les modèles de vue de l'application client (Silverlight),
- « Model » représente le modèle de l'application client,
- « Service » représente l'ensemble des services web partagés à traves la platforme WCF,
- « ServiceApp » représente les services applicatifs offerts par notre application,
- « DTO »Permet de définir l'ensemble des objets de transfert des données,
- « Entity » et « Objet valeur » représentent l'ensemble des objet de domaine,
- « Service Domaine » représente les services de domaine qui fournissent les calculs et les traitements purement spécifiques au domaine métier,
- « IRepository » et « IunitOfWork » représentent les interfaces permettant de communiquer avec le couche infrastructure,
- « Repository » représente l'implémentation des « IRepository » à travres « EntityFramework ».

### 4.3.2 Conception de la structure de présentation

Le maquettage permet, en particulier, de lever les incertitudes liées à la conception. Il permet de refléter les besoins des clients et mettre en avant les difficultés majeures pour les développeurs. Une des méthodes souvent utilisée est celle des maquettes papier sans aucun support informatique. Il s'agit d'une suite d'écrans montrant la navigation à travers l'application ainsi que son utilisation.

Nous avons choisi de présenter les maquettes que nous avons jugées les plus importantes dans notre application.

#### 4.3.2.1 Formulaire d'ajout d'ordre

La figure 4.6 illustre les champs que l'utilisateur doit remplir pour passer un nouvel ordre.

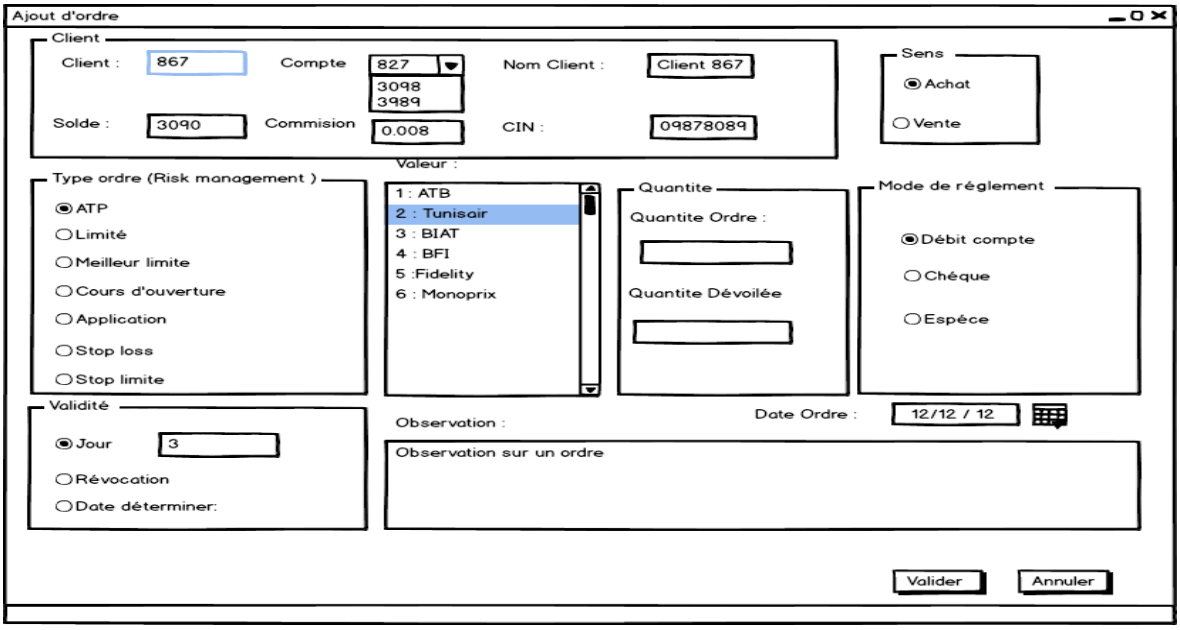



FIGURE 4.6 – Maquette du formulaire d'ajout d'ordre

#### 4.3.2.2 Formulaire d'ajout de liquidité

La figure 4.7 montre les champs que l'utilisateur doit remplir pour ajouter une liquidité dans le cas de « virement interne ».

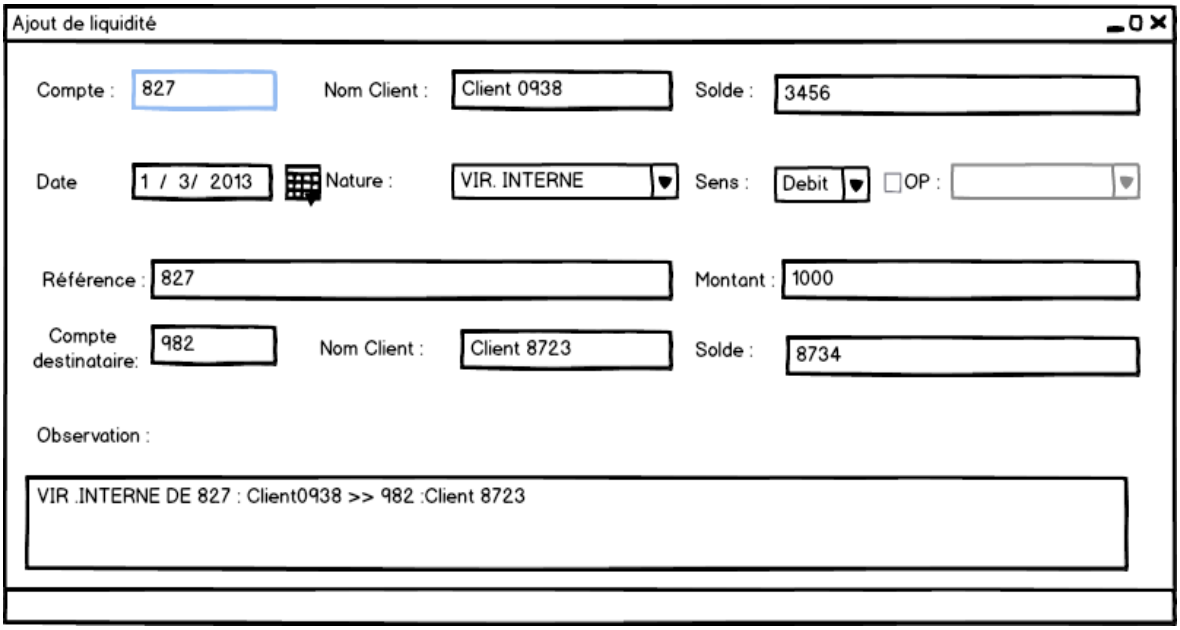



FIGURE 4.7 – Maquette du formulaire de virement interne

#### 4.3.2.3 Formulaire exécution manuelle

La maquette de la figure 4.8 illustre l'interface graphique proposée pour gérer l'exécution manuelle.

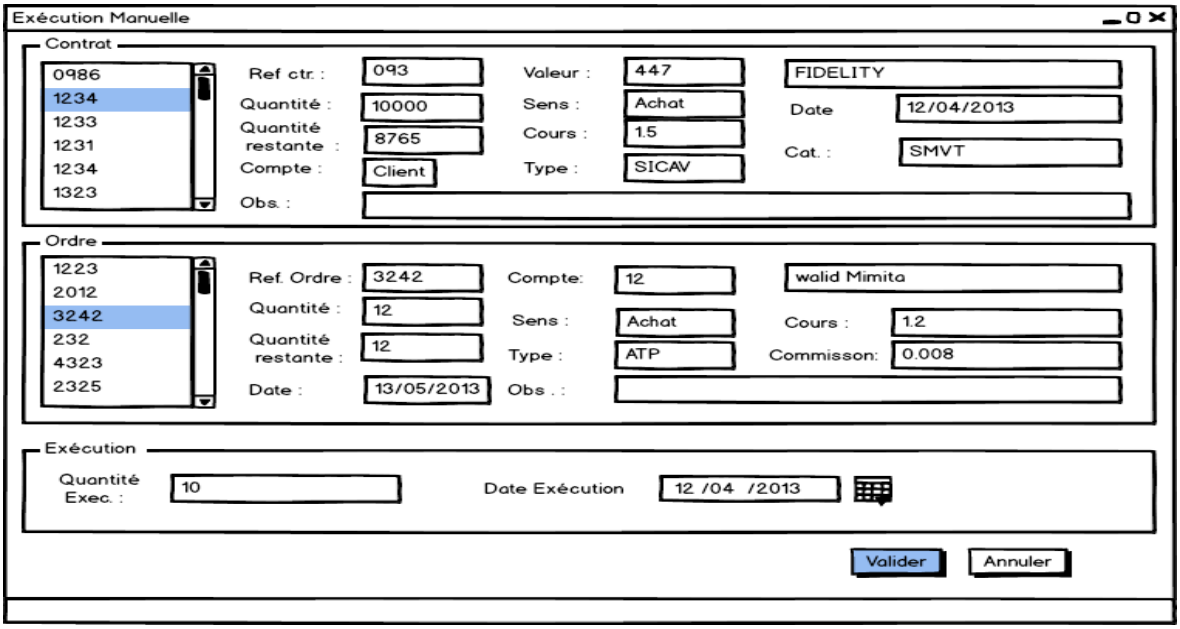



FIGURE 4.8 – Maquette d'exécution manuelle

## 4.4 Conception détaillée

La conception détaillée est la phase ultime de la modélisation. Elle consiste à construire et à documentern d'une façons précise les classes, les tables et les méthodes constituant le codage de la solution tout en prenons en compte les patrons de conception sélectionnée dans la phase de la conception générique.

Vu l'importance capitale de la couche domaine dans une architecture DDDD, nous allons faire recours à des diagrammes de classes pour décortiquer les différents éléments qui la composent.

### 4.4.1 Diagramme de classe «ClientBoundedContext»

Le diagramme de la figure 4.9 illustre les class de l'agregat « ClientBoundedContext ».

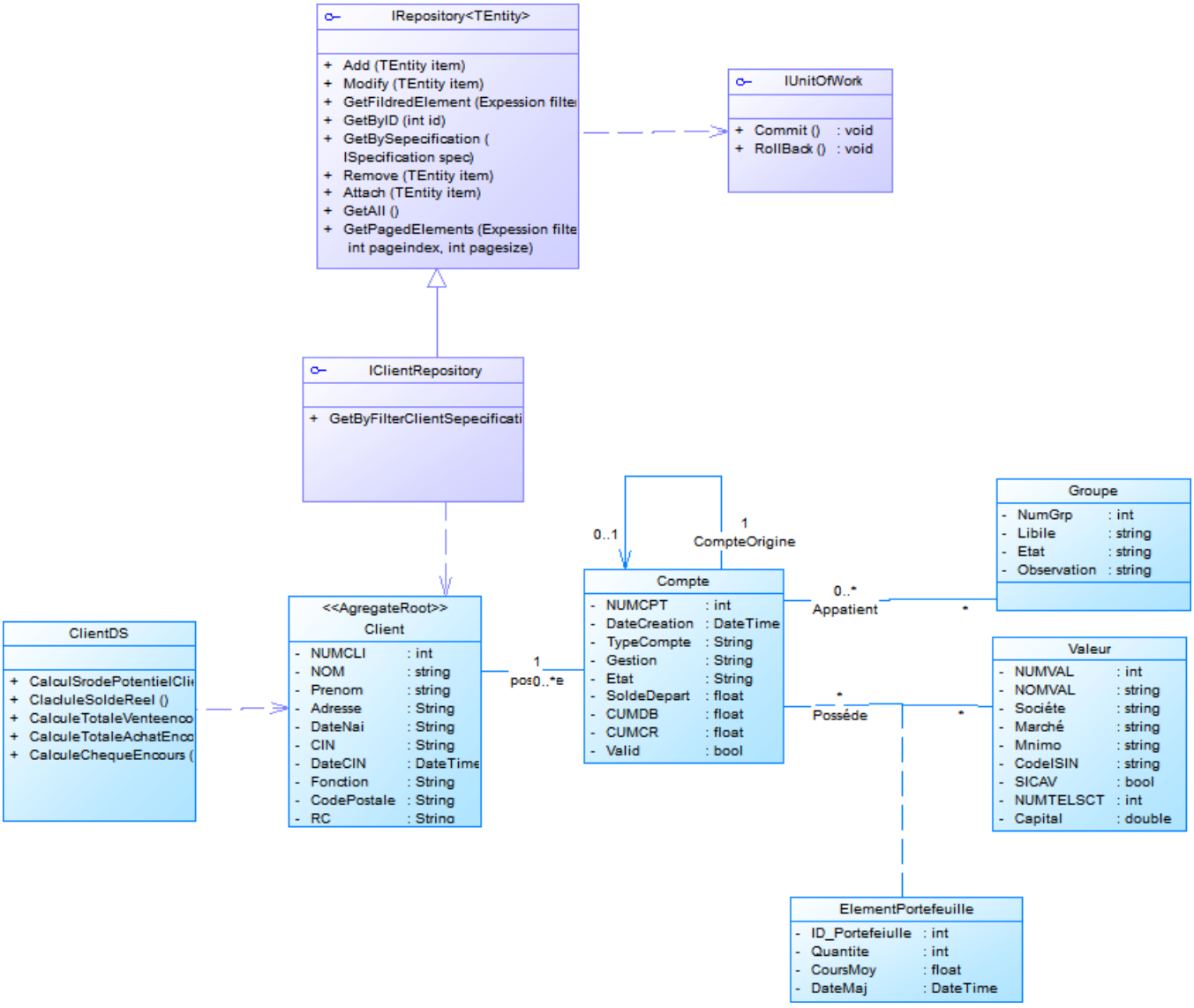



FIGURE 4.9 – Diagramme de classe «ClientBoundedContext»

Dans le tableau 4.2, un commentaire détaillée du diagramme de classe relatif à l'agrégat «ClientBoundedContext» est relaté.

| **Class / interface** | **Description** |
|---|---|
| Client | Cette classe contient les informations de base d'un client. La classe « Client » représente la racine de l'agrégat client «AgregateRoot » elle est gérée par un « ClientReprository ». |
| Compte | Cette classe contient les informations de base d'un compte. l'accès à cette classe est autorisé qu'à travers la racine de l'agrégat Client. |
| Groupe | Cette classe encapsule toutes les informations nécessaires pour un groupe de comptes. |
| ElementPortefeuille | Cette classe constitue la composition d'un portefeuille. |
| ClientDS | Cette classe représente le service de domaine du module client, elle contient toutes les opérations de calcul spécifique au domaine de ce module à savoir le calcul des soldes et des mentants des opérations effectuées par les clients. |
| IUnitOfWork | C'est une interface qui permet la communication avec la base des données, assure la persistance des objets, minimise le coût d'accès à la base de données tout en gardant la trace de toute action effectuée. |
| IRepository | C'est une interface générique contenant des méthodes permettant la communication avec la base de données et assure la persistance des objets par l'intermédiaire d'IUnitOfWork. |
| ICleintRepository | L'interface « IClientRepository » assure la communication avec l'entrepôt des clients. |

TABLE 4.2 – Tableau descriptif des classes de ClientBoundedContext

### 4.4.2 Diagramme de classe « OrdreBoundedContext»

Le diagramme de classe relatif au contexte Ordre est représenté par la figure 4.10.

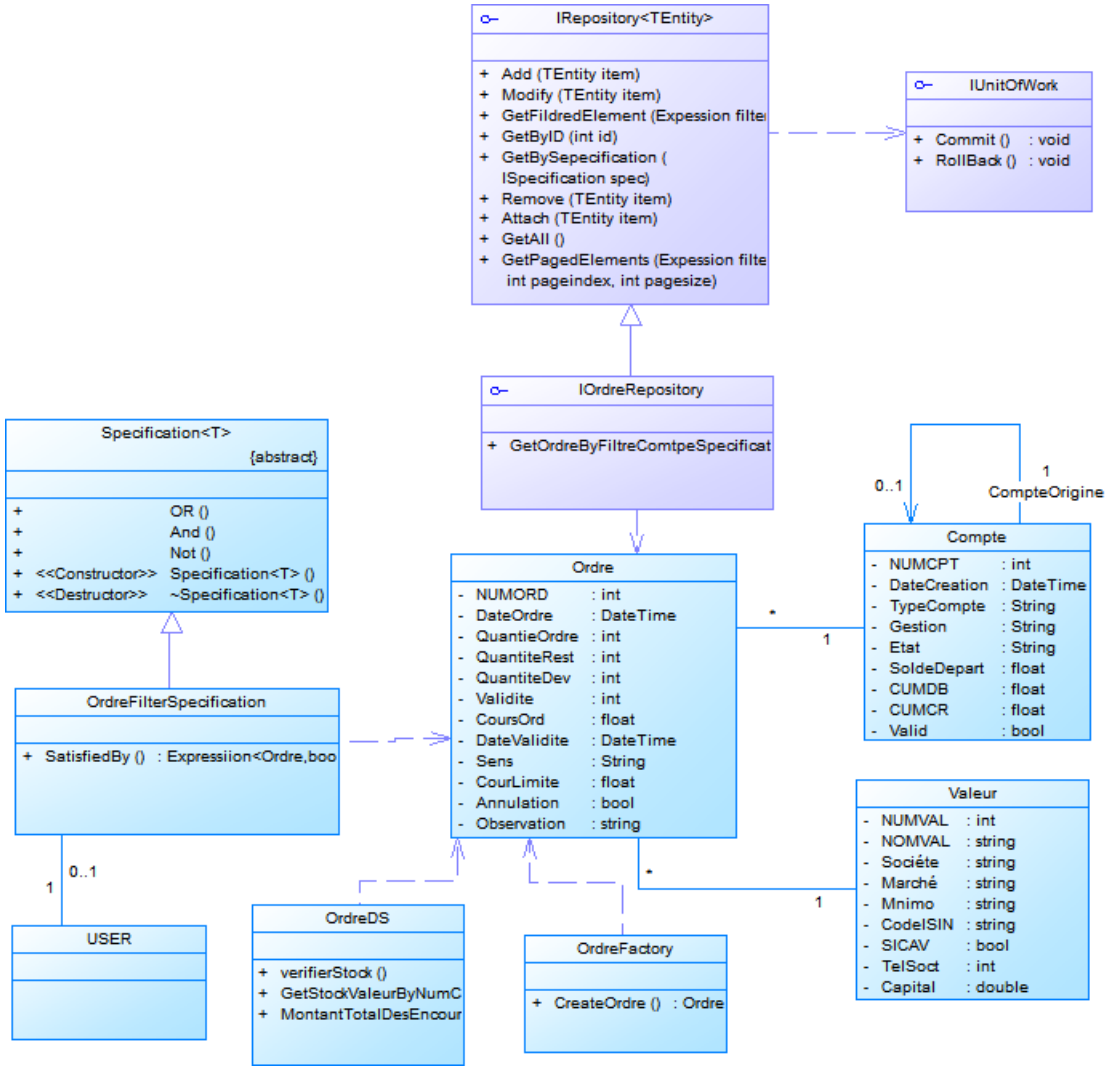



FIGURE 4.10 – Diagramme de classe «OrdreBoundedContext»

Le tableau 4.3 donne une description détaillée du diagramme de classe relatif à l'agrégat «OrdreBoundedContext».

| **Class / interface** | **Description** |
|---|---|
| Ordre | Cette classe contient les informations de base d'un ordre d'achat ou de vente. La classe ordre est gérée par un « ClientReprository » et « OrdreFactory ». |
| Valeur | Cette classe contient les informations nécessaires pour une valeur mobilière. |
| IOrdreRepository | L'interface « IOrdreRepository » et assure la communication avec l'entrepôt des ordres. |
| OrdreDS | Cette classe représente le service de domaine du module ordre, elle encapsule la logique spécifique au domaine à savoir les vérification de stock et calcule des quantités des ordres encours. |
| OrdreFactory | C'est une classe qui s'occupe de la création des ordres. |
| OrderFiterSpecification | C'est class qui encapsule la logique de filtre sur les ordres visible pour un utilisateur du système. |
| USER | La class USER encapsule tous les informations d'un utilisateur de système. |

TABLE 4.3 – Tableau descriptif des classes de OrdreBoundedContext

### 4.4.3 Diagramme de classe «ExecutionBoundedContext»

Le diagramme de classe relatif au contexte Execution est représenté par la figure 4.11.

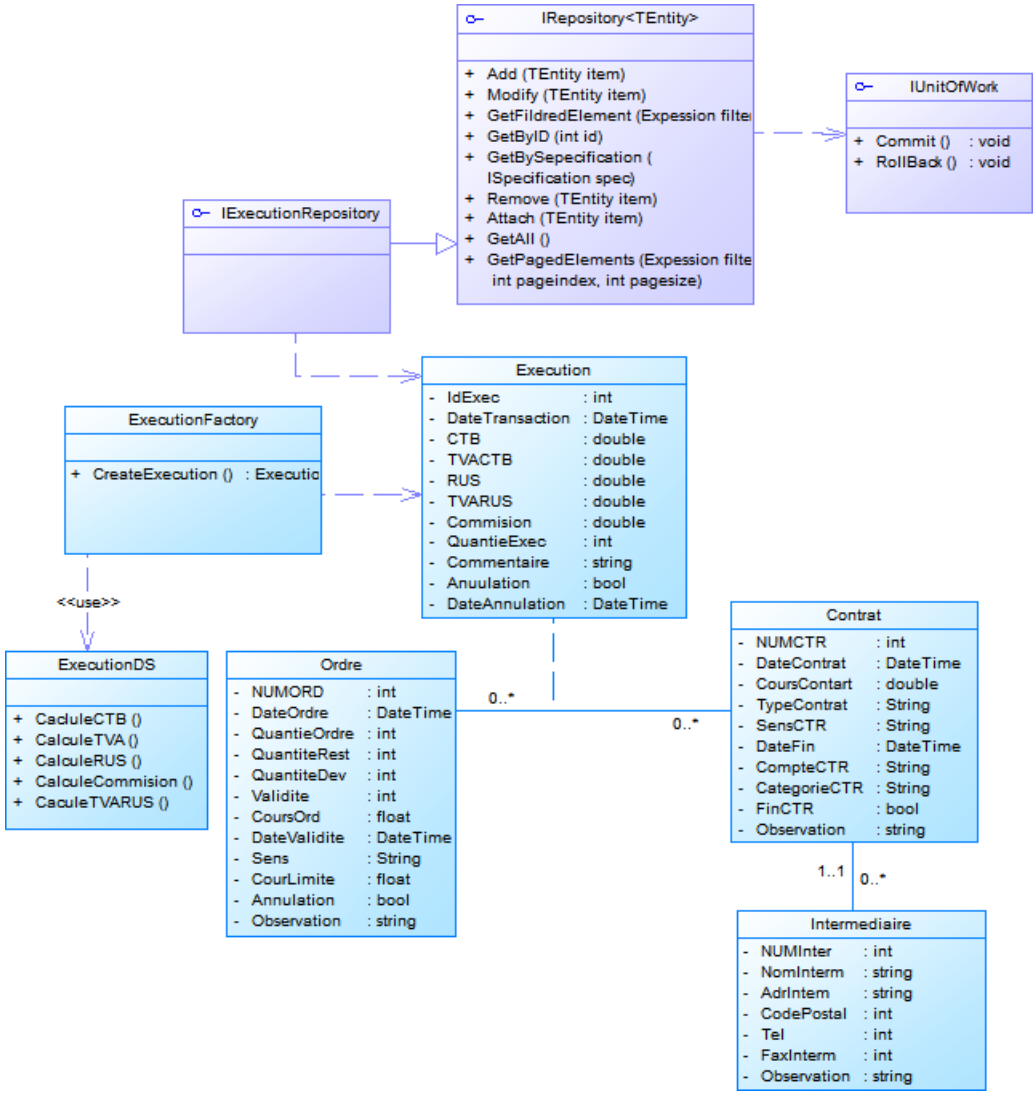



FIGURE 4.11 – Diagramme de classe «ExecutionBoundedContext»

Dans le tableau 4.4, le diagramme de classe relatif au context «ExecutionBoundedContext» est commenté.

| **Class / interface** | **Description** |
|---|---|
| Execution | Cette classe contient les informations indispensables pour une exécution d'un ordre pour le profit d'un contrat. |
| Conrtat | Cette classe regroupe l'ensemble des informations nécessaires pour un contrat d'achat ou de vente d'une valeur mobilière donnée. |
| Intermediaire | Cette classe encapsule toutes les informations nécessaires pour un intermédiaire en bourse. |
| ExecutionDS | Cette classe représente le service de domaine du module Exécution, elle encapsule la logique de spécifique ce module à savoir les calcule des commissions, des taxes et le retenu à la source. |
| ExecutionFactory | C'est une classe qui s'occupe de la création des objets de la classe « Execution», elle fait recours à la classe ExecutionDS pour calculer les informations nécessaires. |
| IExecutionRepository | L'interface « IExecutionRepository » et assure la communication avec l'entrepôt des exécutions |

TABLE 4.4 – Tableau descriptif des classes de ExecutionBoundedContext

### 4.4.4 Diagramme de classe «LiquiditeBoundedContext»

La figure 4.12 permet d'illustrer le diagramme de classe relatif au contexte Liquidité.

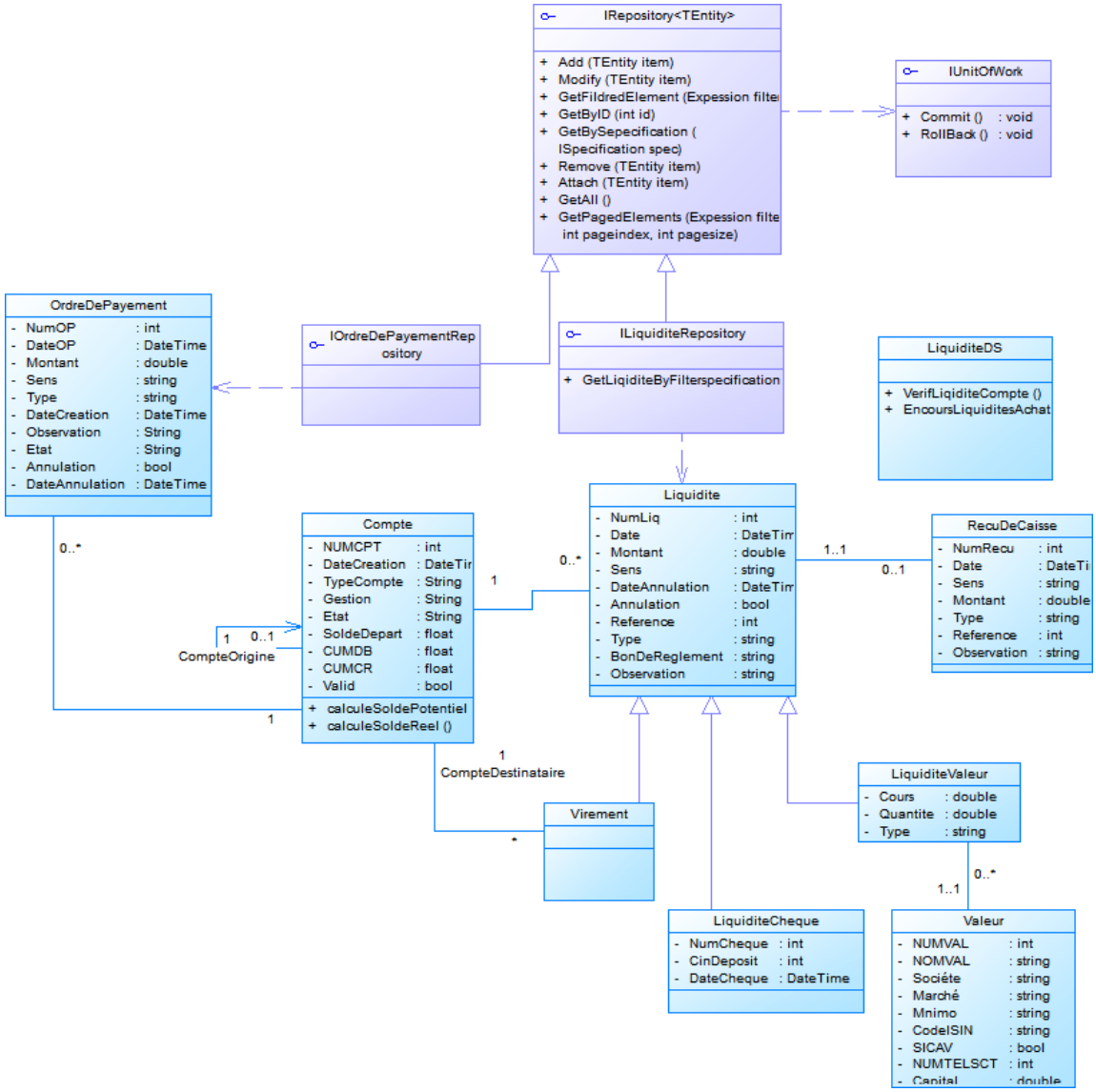



FIGURE 4.12 – Diagramme de classe «LiquiditeBoundedContext»

Le tableau 4.5 est une description détaillée du diagramme de classe relatif au contexte «LiquiditeBoundedContext».

| Class / interface | Description |
|---|---|
| Liquidite | Cette classe regroupe l'ensemble des informations nécessaires pour enregistrer une liquidité. |
| OrdreDePayement | Cette classe encapsule toutes les informations nécessaires pour un ordre de payement passer par un commercial. |
| LiquiditeDS | Cette classe représente le service de domaine du module Liquidité, elle encapsule la logique de spécifique ce module comme la vérification des liquidités et le calcul des liquidités en cours. |
| ILiquiditeRepository | L'interface « ILiquiditeRepository » et assure la communication avec l'entrepôt des Liquidités. |
| IOrdreDePaymentRepository | L'interface « IOrdreDePaymentRepository » et assure la communication avec l'entrepôt des OrdreDePayements. |

TABLE 4.5 – Tableau descriptif des classes de LiquiditeBoundedContext

## 4.5 Conclusion

A travers ce chapitre, nous avons présenté une vue conceptuelle de notre projet. Nous avons aussi énoncé l'architecture logicielle de notre système via un modèle logique générique élaboré sous forme d'un diagramme de composant. A ce stade, nous pouvons passer à la réalisation de la solution qui fera l'objet du chapitre suivant.

Chapitre 5

# Réalisation

## 5.1 Introduction

Ce chapitre constitue le dernier volet du processus 2TUP, il décrit la phase de codage et de test de l'application comme l'illustre la figure 5.1. Il permet, en effet, de présenter dans un premier lieu, l'environnement matériel et logiciel utilisé, pour exposer enfin un aperçu sur le travail réalisé à travers quelques captures d'écrans commentées.

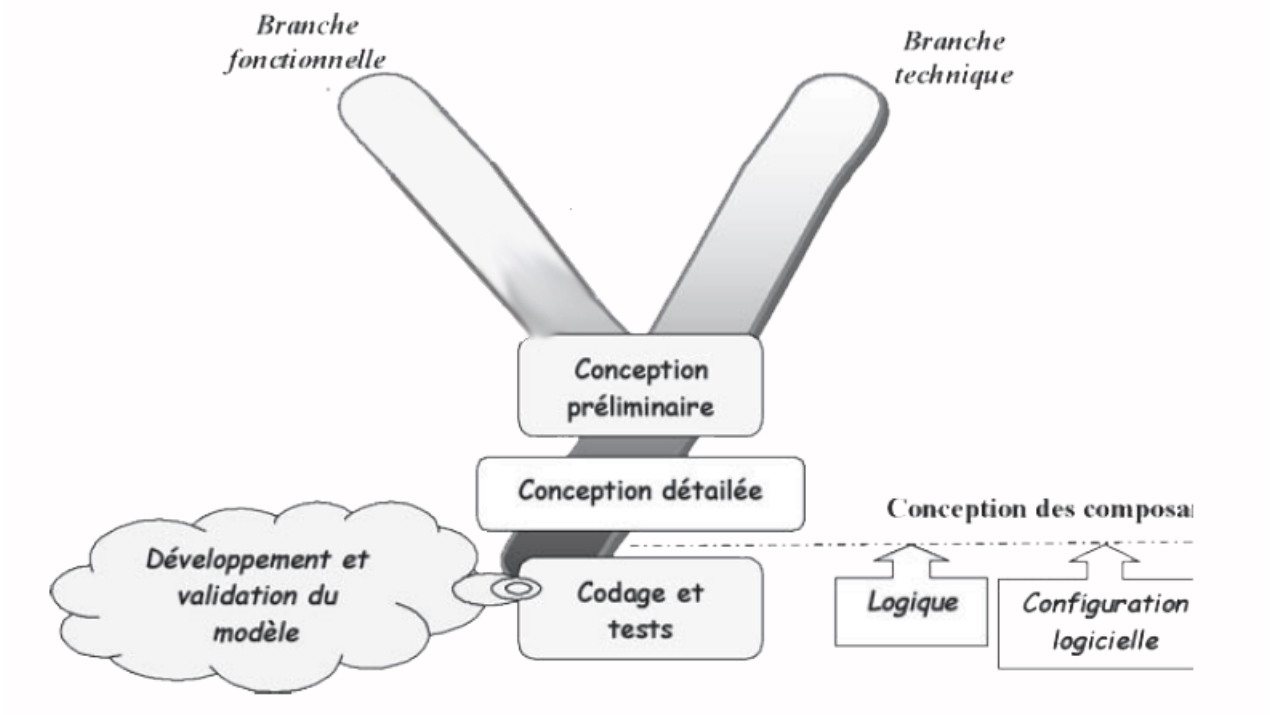



FIGURE 5.1 – Réalisation

## 5.2 Environnement matériel et logiciel

Cette partie présente les ressources matérielles et logicielles utilisées dans le cadre de ce projet.

### 5.2.1 Environnement matériel

Le présent travail est réalisé sur un Micro-ordinateur ayant la configuration suivante décrite dans le tableau 5.1.

| | |
|---|---|
| Système d'exploitation | Windows Seven Ultimate |
| Mémoire centrale | 2 GO |
| Processeur | Intel® Core™ 2 Duo T6500 2.10GHz |
| Disque Dur | 160 GO |

TABLE 5.1 – Environnement matériel

### 5.2.2 Environnement logiciel

Nous consacrerons cette section à la présentation des diérentes technologies et outils logiciels utilisés pour le développement de notre application.

#### 5.2.2.1 Environnements de développement intégré « IDE »

Pendant tout le cycle de codage et de développement, nous avons utilisé l'IDE commun qui est incontournable pour le développement des applications basées sur les technologies Microsoft, à savoir, Visual Studio 2010. Il fournie des outils qui facilite la création de solutions web et bureautiques basées sur le Framework .NET faisant appel à plusieurs langages comme C#, VB, F# et C++.

#### 5.2.2.2 Outil de modélisation « Sybase PowerAMC »

Pour répondre à nos besoins et à nos exigences, dont les plus importants la réutilisabilité, la modularité et l'extensibilité, nous avons choisi de modéliser notre système avec le langage de modélisation UML en utilisant l'outil « Sybase PowerAMC ».

#### 5.2.2.3 Outil de gestion de versions

TFS « Team Foundation Server » [13] est une forge logicielle éditée par Microsoft permettant la gestion des sources, la gestion des builds, le suivi des éléments de travail, la planification, la gestion de projet et l'analyse des performances afin d'augmenter la productivité.

#### 5.2.2.4 SQL Server

Microsoft SQL server est un système de gestion de base de données (SGBD) développé par Microsoft. Par souci d'incompatibilité avec les applications de CybEx, nous avons choisi d'utiliser la version 2008 de SQL server.

#### 5.2.2.5 Unity

« Microsoft Unity Application Block »[10] est un framework qui fait partie des « Patterns & Practices » de Microsoft. C'est un outil suggéré par Microsoft pour un développement plus propre. Il offre un conteneur d'injection de dépendances, de poids léger, extensible et aide à la construction d'architectures découplées.

#### 5.2.2.6 PollingDuplexHttpBinding

Le binding est un élément de base dans l'architecture WCF, il permet de décrire le protocole utilisé pour se connecter à un point de communication d'un service WCF en spécifiant chaque élément de la pile de communication : Transaction, Fiabilité, Sécurité, Encodage, Transport.

Afin d'assurer une communication bidirectionnelle entre le navigateur web du client et un service déployé sur le serveur, nous avons fait recours à un nouveau type de binding conçu spécialement pour Silverlight nommé PollingDuplexHttpBinding.

Un service duplex [14] maintient alors un canal de rappel au client Silverlight, qui permet au service d'effectuer des appels vers le client.

## 5.3 Travail réalisé

La présente section décrit la phase d'implémentation du projet à travers une description de la phase de préparation du projet ainsi qu'un aperçu sur les interfaces graphiques que nous avons jugés les plus intéressantes pour illustrer notre application.

### 5.3.1 Préparation du projet

Afin d'assurer des critère d'extensibilité et de réutilisabilité, nous avons préparé dans premier lieu, les différentes couches de notre projet (Présentation, ServiceDistibuer, Application, Domaine et Infrastructure) à travers une implémentation de projets « Core » et qui peuvent être utilisés dans le cadre de prochains projet au sein de « Cybex Solution ». Ces projets réalisés suivant l'approche DDD sont décrit dans ce qui suit :

#### 5.3.1.1 Domain.Core

Comme son nom l'indique, ce projet constitue le coeur de la couche domaine. Il contient les différentes classes et interfaces indispensable pour cette couche comme IRepository, IUnitOfWork ainsi que l'implémentation du design pattern « specification ».

#### 5.3.1.2 Infrastucture.Core

Ce projet contient l'implémentation des patrons de conception Repository et UnitOfwork via des classes génériques tout en se basant sur EntityFramework.

#### 5.3.1.3 CorssInfrastucture.Core

Ce projet représente le coeur de la couche transverse « CrossInfrastructure ». il contient les interfaces qui assurent la communication avec cette couche.

### 5.3.2 Interfaces Homme/Machine

Les interfaces Homme/Machine constituent un facteur important dans la réussite ou l'échec d'une application. Cette contribution est d'autant plus importante lorsqu'il s'agit d'une application RIA. Ainsi, nous essayerons dans cette partie de présenter par module les interfaces les plus représentatives de notre application.

### 5.3.3 L'authentification

La protection est nécessaire afin de respecter la confidentialité des données sensibles. Pour cela, lors du lancement de l'application, une fenêtre d'identification s'affiche. Après l'authentification, l'utilisateur pourra accéder aux différents modules de l'application. La figure 5.2 représente le formulaire de connexion.

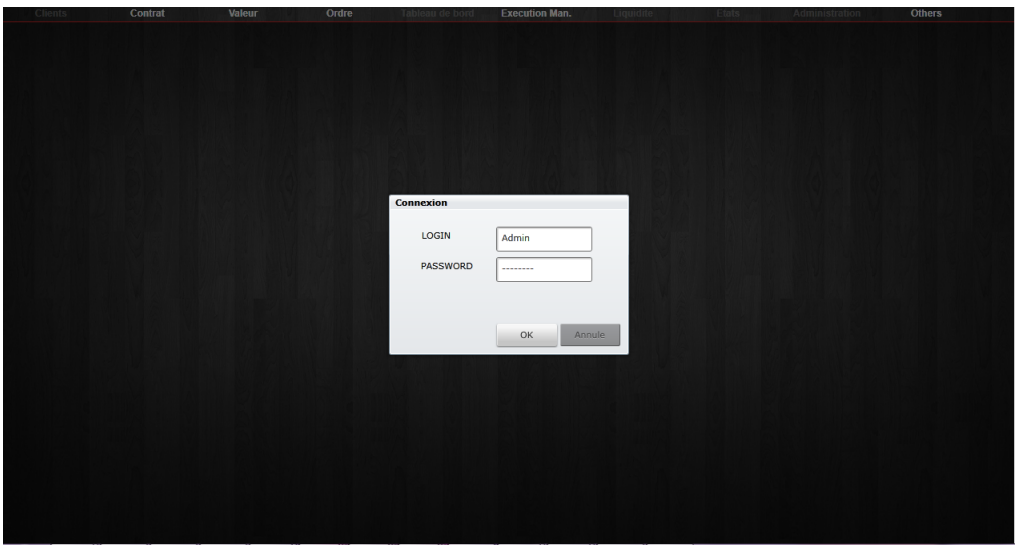



FIGURE 5.2 – Authentification

### 5.3.4 Module de gestion des valeurs

Ce module fait partie des modules servant à la création du référentiel de la solution.il offre les fonctions d'ajout et d'édition des Valeurs. Il propose également la consultation par un double clic sur la ligne choisie. La figure 5.3 représente le formulaire de modification d'une valeur.

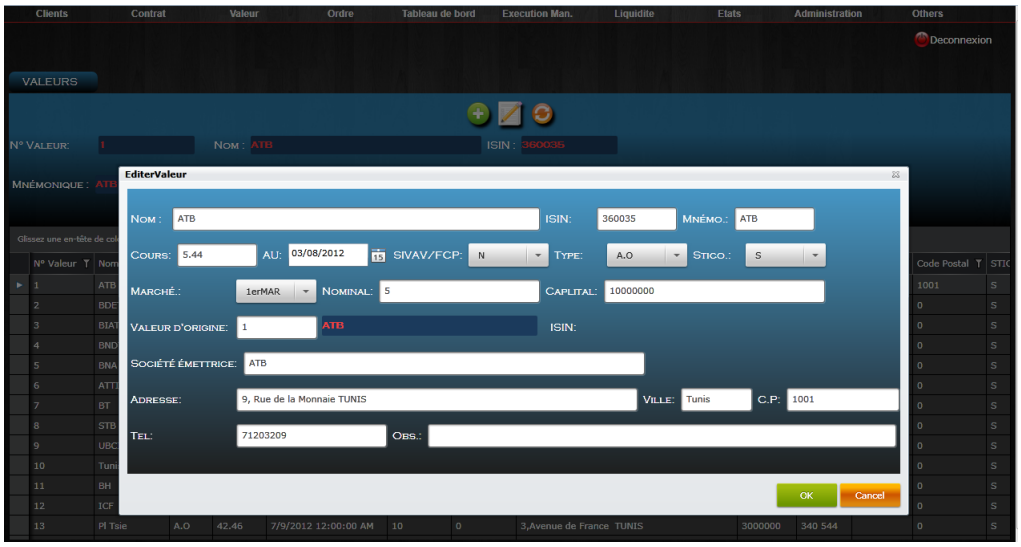



FIGURE 5.3 – Module Valeurs

### 5.3.5 Module de gestion des contrats

Ce module propose, comme ullistre la figure5.4, la création et l'édition d'un contrat d'achat ou de vente. Ce contrat sera ensuite exécuté au moyen du module exécution manuelle. Le module contrat permet aussi la consultation et le filtrage des contrats. Les contrats exécutés sont automatiquement retirés de la grille. Lors de la saisie d'un contrat, des informations essentielles sont saisies, comme la date et le type de contrat (Normal, Sicav, FCP, Sans Mouvement de Fonds, etc.). Ces données sont importantes car elles influenceront, lors de l'exécution, le calcul du montant net de la transaction.

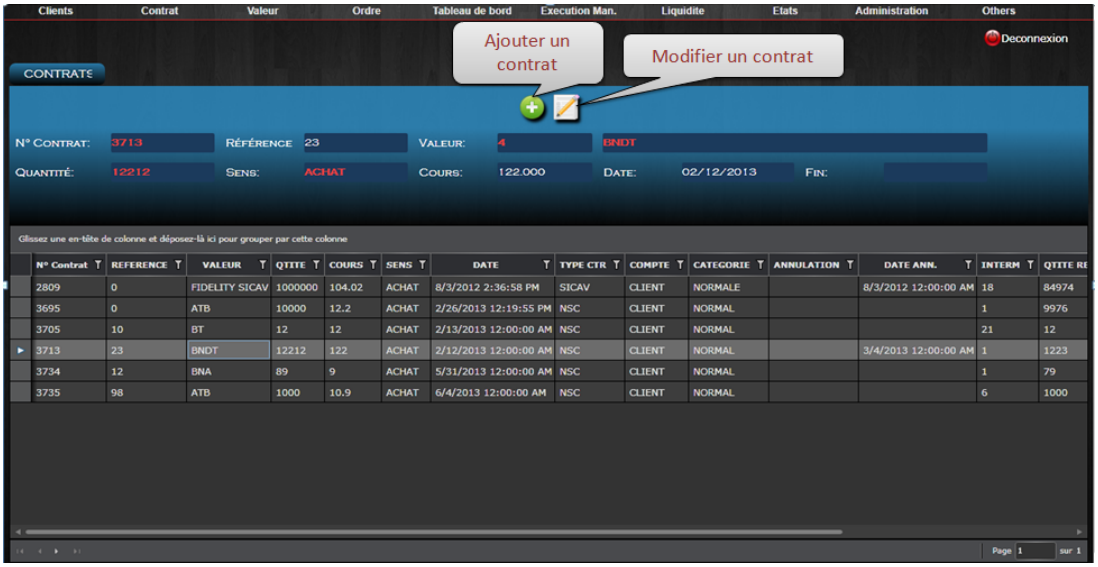



FIGURE 5.4 – Module Contrats

### 5.3.6 Module de gestion des clients

Ce module, illusté dans la figure 5.5, fait partie du référentiel de la solution. Il permet de préciser les informations essentielles des clients comme le nom, la nationalité, la gestion, le commercial qui prendra en charge le client et les informations sur les comptes comme le type (ordinaire, sicav, CEA, FCP, etc.), l'état et la commission à appliquer.

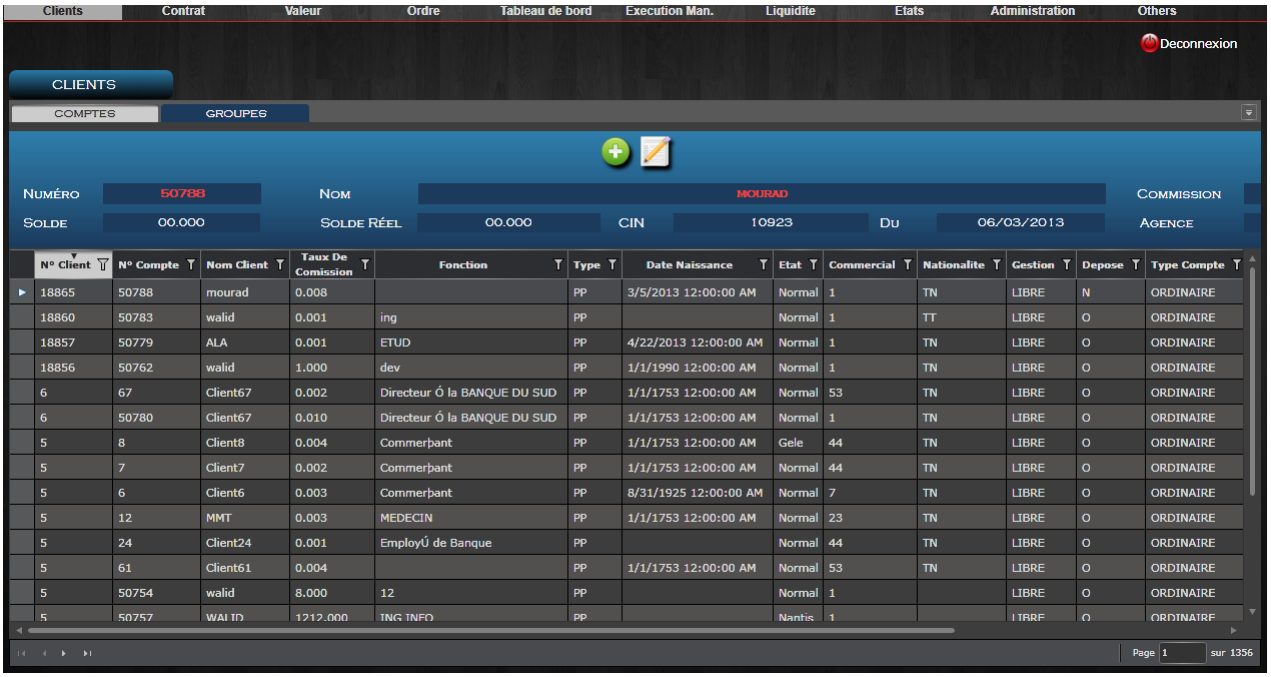



FIGURE 5.5 – Module Client

Ce module permet aussi la possibilité de regrouper des comptes sous un même groupe comme

le montre la figure 5.6.

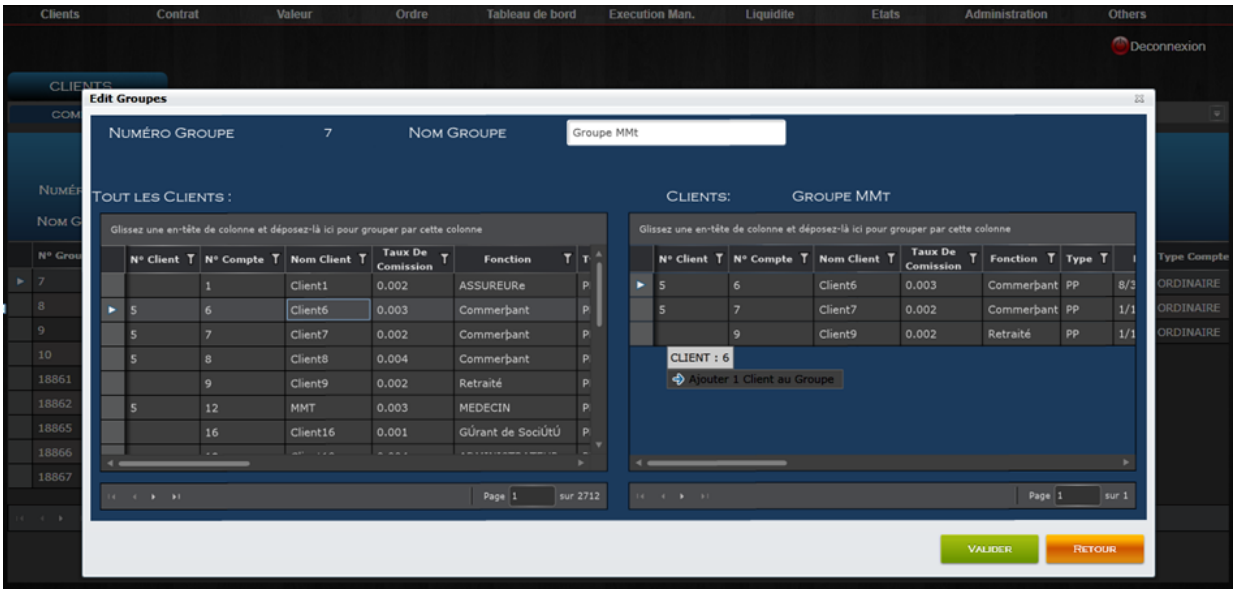



FIGURE 5.6 – Affectation dans un Groupe

Afin de visualiser plus d'informations sur un client, nous avons ajouté une sorte de complément pour le module client qui s'intitule « Tableau de bord » et qui est illustré par la figure 5.7.

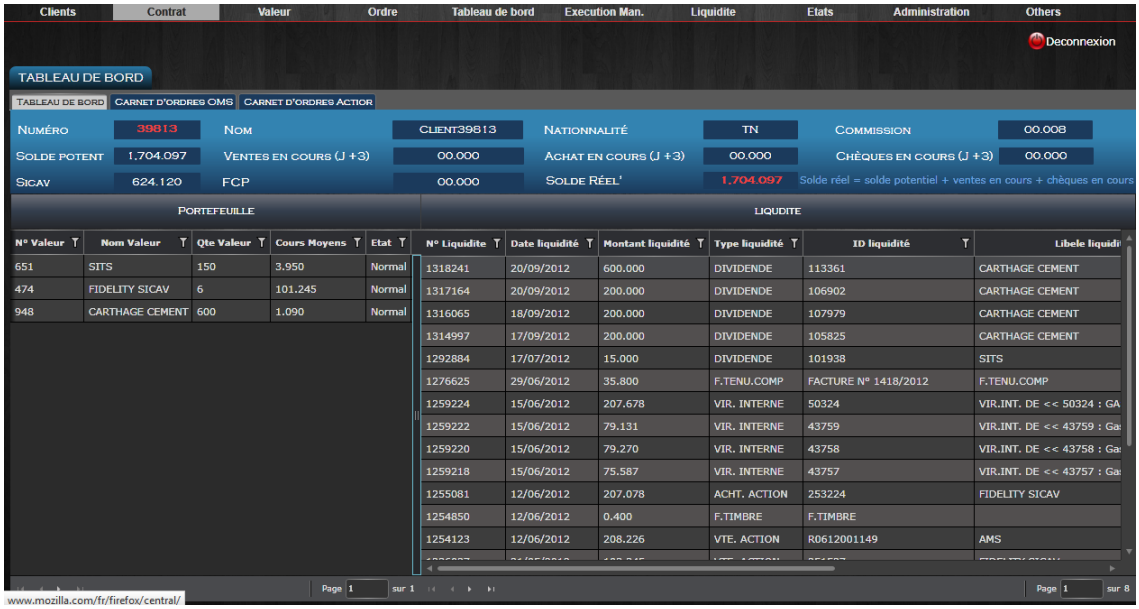



FIGURE 5.7 – Tableau de bord

### 5.3.7 Module de gestion des ordres

Ce module propose la création, la consultation et la recherche des ordres. Il permet aussi aux superviseurs de voir en temps réel, les ordres passés par leurs commerciaux. Par simple clic sur

le bouton « imprimer » la ligne d'ordre selectionnée sera imprimée.La figure 5.8 représente le carnet d'ordre.

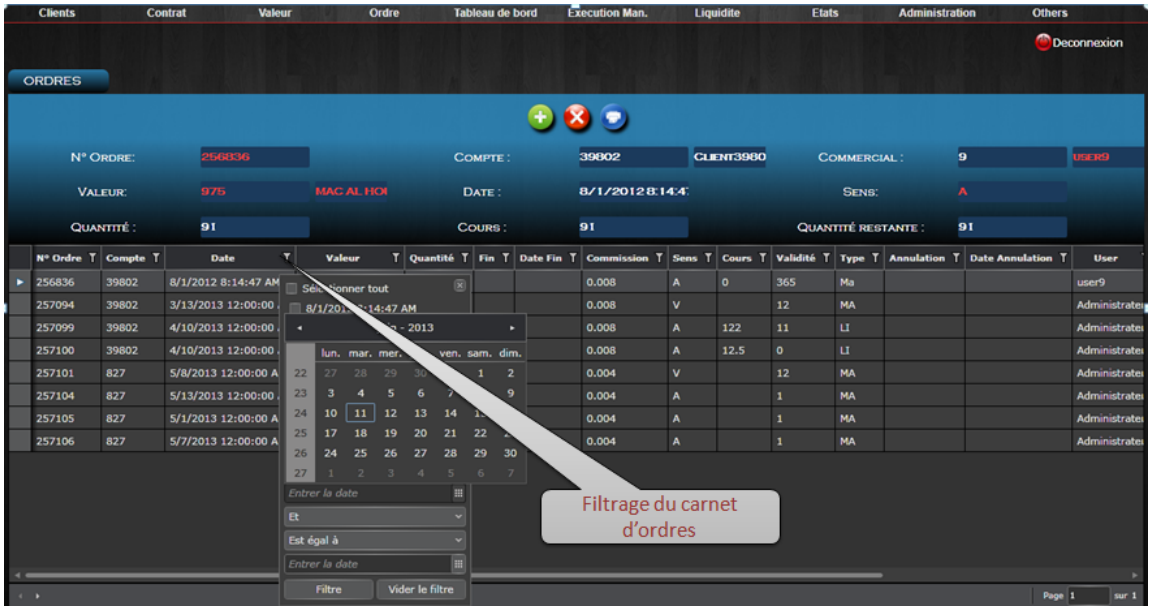



FIGURE 5.8 – Carnet d'ordres

La figure 5.9 représente le formulaire d'ajout d'un ordre.

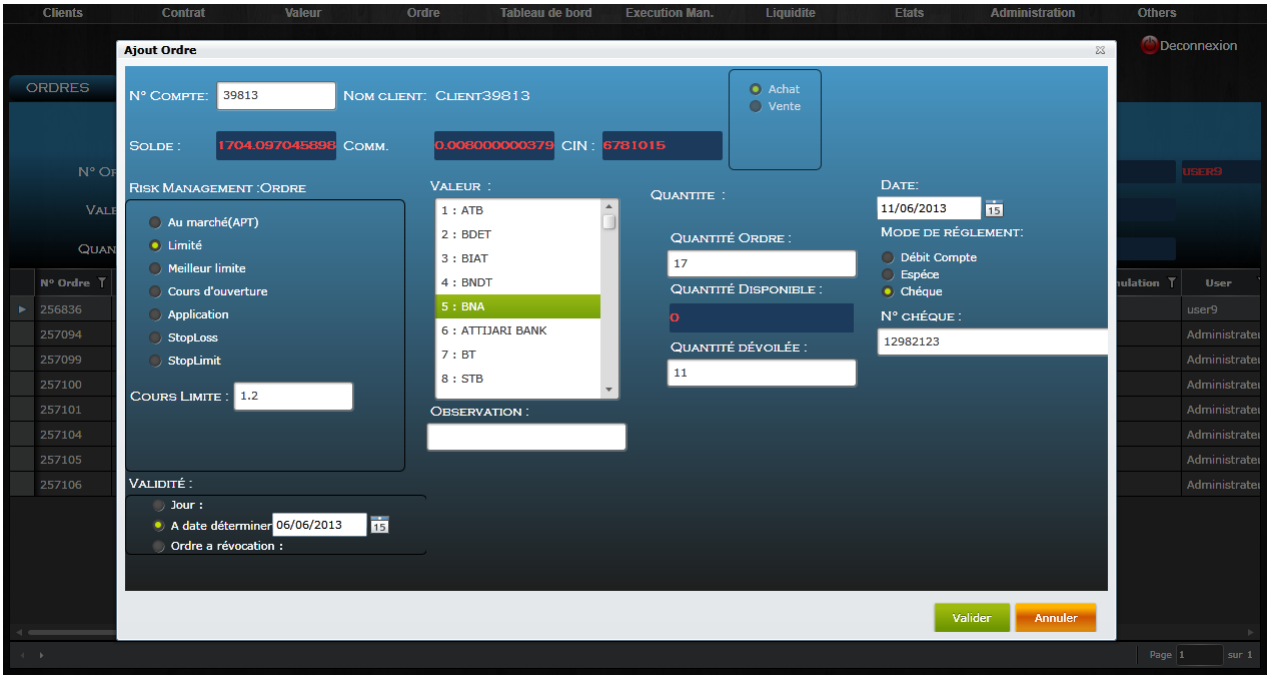



FIGURE 5.9 – Formulaire d'ajout d'un ordre

### 5.3.8 Exécution mannuelle

L'exécution manuelle permet de mettre face à face un contrat et un ou plusieurs ordres préalablement saisis.Lors de la sélection d'un contrat , les ordres associés (valeur, sens, type, etc.) s'afficheront dans la liste des ordres. Une sélection de l'ordre choisi permet de le saisir automatiquement. Il ne reste plus qu'à saisir la quantité de la transaction puis d'appuyer sur le bouton valider. tel qu'illustré dans la figure 5.10.

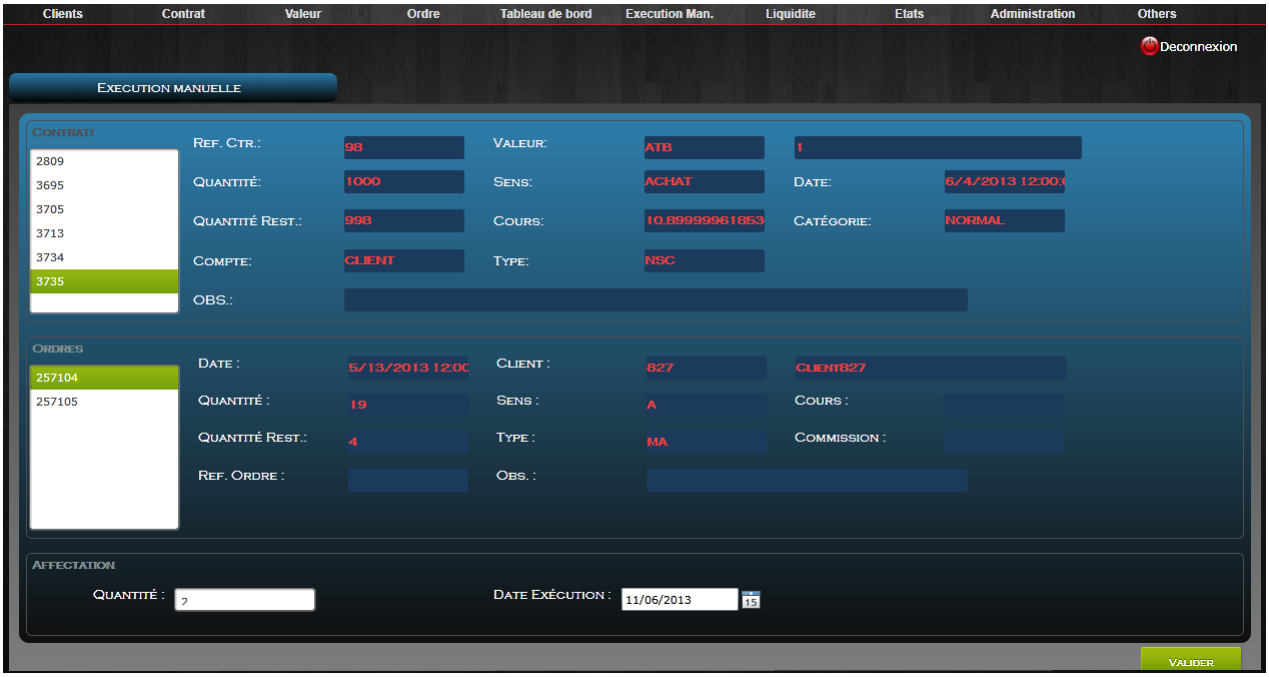



FIGURE 5.10 – L'execution mannuelle

### 5.3.9 Module de gestion des liquidités

Ce module décrit par la figure 5.11, propose la saisie, la consultation et la recherche d'écritures des liquidités comme les Versements et les retraits de différents types (espèces, chèques, Vir. Postal, etc.).

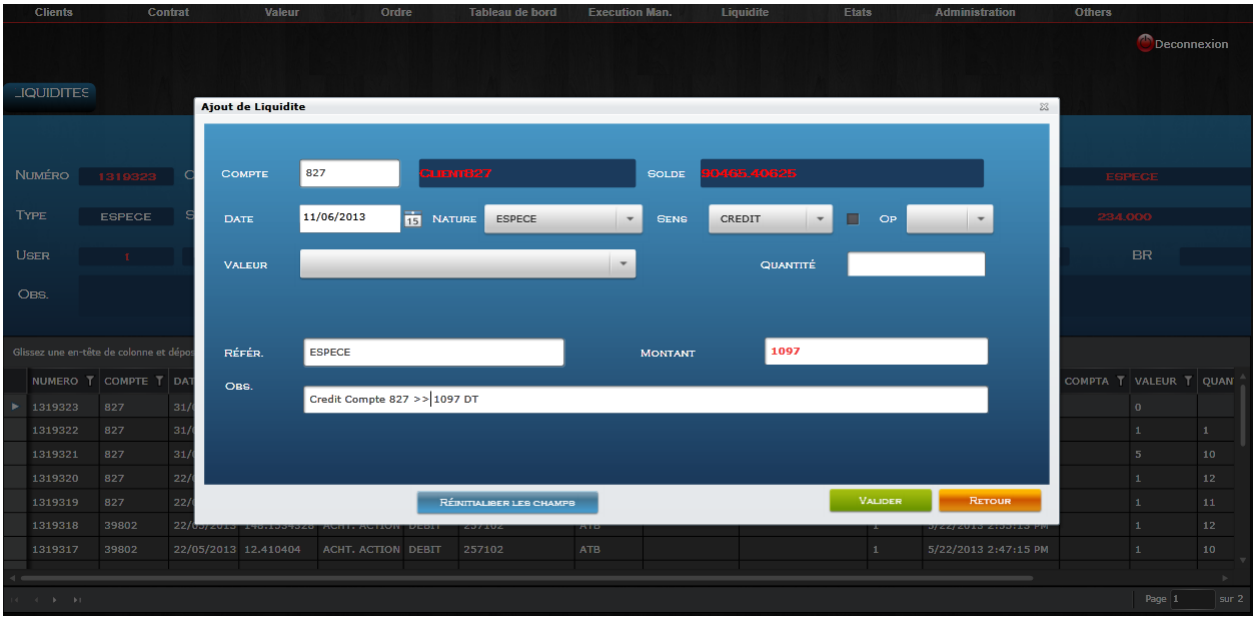



FIGURE 5.11 – Saisie de Liquidité

La saisie peut se faire aussi par la sélection d'un Ordre de Payement préalablement saisie grâce au module Ordre de Payement que nous présenterons dans la fenêtre de la figure 5.11.

Pour des raisons d'organisation et de sécurité, le client souhaitant retirer de l'argent, s'adresse d'abord à son commercial qui lui délivre un Ordre de Payement, Ensuite, le client se présente muni de son OP à la caisse où les informations seront importée sous le module liquidité.

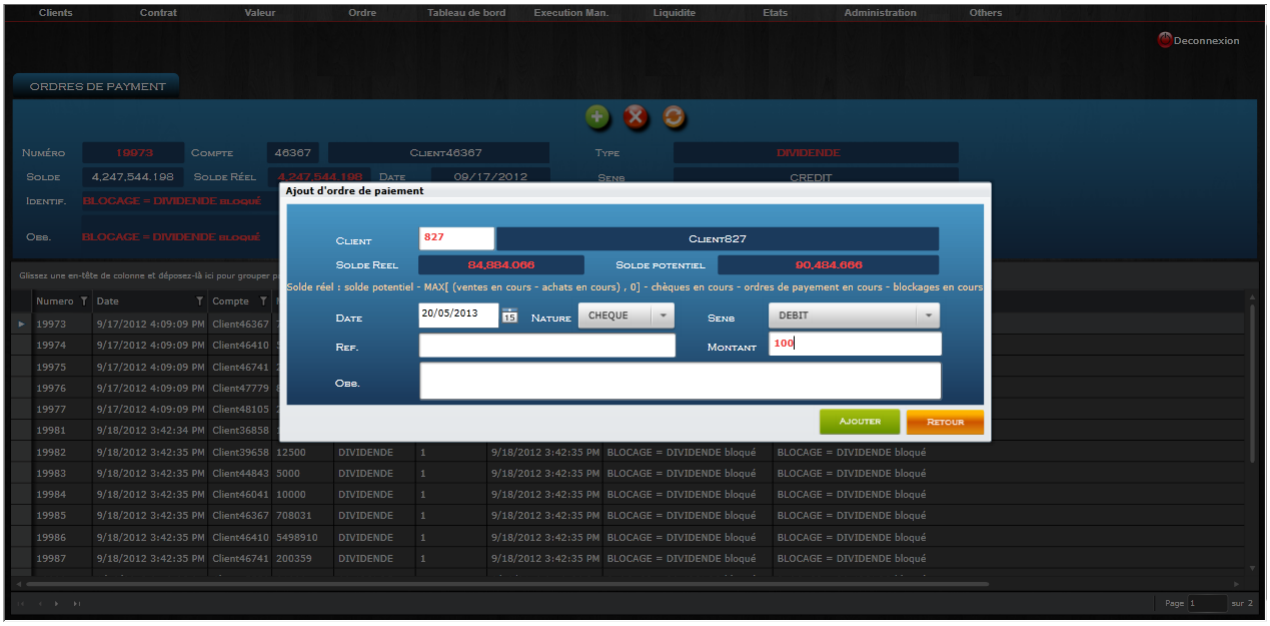



FIGURE 5.12 – Ordre de Payement

Le module liquidité offre aussi la possibilité d'intégrer automatiquement plusieurs lignes au moyen d'une fiche excel comme le montre la figure 5.13.

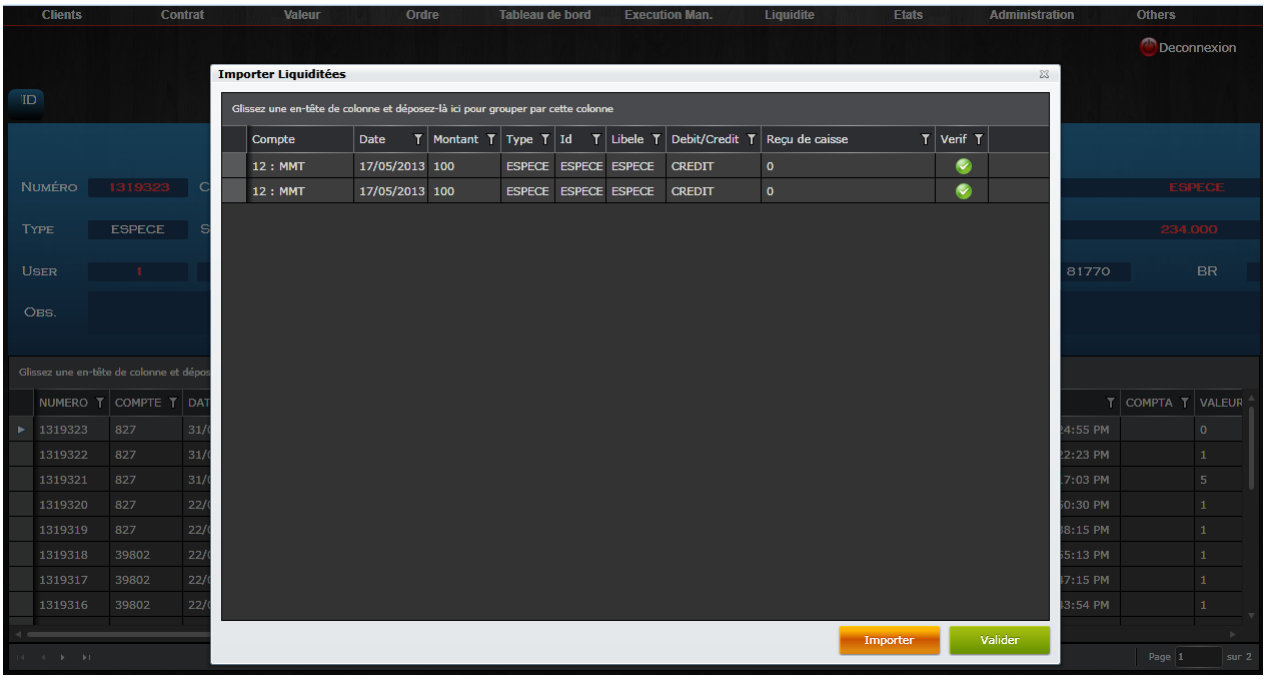



FIGURE 5.13 – Import Excel

## 5.4 Conclusion

Dans ce chapitre nous avons présenté la phase d'implémentation de notre projet. Après avoir défini les technologies et les outils utilisés, nous avons décrit les fonctionnalités de notre application en présentant des captures d'écran.

# Conclusion générale

Dans un contexte de fortes concurrences et d'augmentation du volume de données traitées, les systèmes de gestion des intermédiaires en bourse se trouvent aujourd'hui devant plusieurs défis à confronter afin de répondre aux exigences des commerciaux, des administrateurs et des agents de Back-office.

C'est dans ce cadre que s'inscrit ce projet de fin d'études, il nous a été confié de contribuer à la mise en place d'un système de gestion d'un intermédiaire en bourse et plus précisément au développement du volet Back-office, offrant une gestion complète des référentiels, des ordres, des liquidités et de l'exécution manuelle.

Durant ce stage, nous avons eu l'opportunité de mettre en application les différentes connaissances acquises durant notre cursus à l'ENSI. De plus, nous avons eu l'occasion d'acquérir de nouveaux aspects théoriques et technologiques, à savoir, les notions du métier des intermédiaires en bourse, les principes du Domain Driven Design et la manipulation des nouvelles technologies comme WCF, Silverlight et EntityFramework.

Entre autre, nous avons tenté, à travers ce projet, de démontrer l'importance d'une approche de conception dirigé par le domaine DDD et d'une méthode de développement basée sur le langage de modélisation UML, qui est la méthode 2TUP.

En guise de perspectives, nous pouvons citer comme améliorations et extensions pour notre système, la manipulation d'autres types d'opérations boursières comme les opérations sur titres « OST » , les opération publiques d'achat ou de vente « OPV-OPA » et l'intégation d'un module pour la gestion des SICAV afin de cerner tous les aspects du métier d'intermédiation en bourse.

# Bibliographie


[1] Abel Avram et Floyd Marinescu, Domain Driven Design vite fait, 1er Editions, C4Media, Etats Unis, 2006

[2] Pascal Roques et Franck Vallée, UML 2 en action De l'analyse des besoins à la conception, 4e édition, Eyrolles Edition, France, 2007

[3] Olivier DAHAN, M-V-VM et Silverlight – De la théorie à la pratique, 1er Editions, E-Naxos, France, 2010.

[4] Mamta Dalal et Ashish Ghoda, XAML Developer reference, 1 er Editions, Russell Jones, Etats Unis, 2011.

[5] http ://www.bvmt.com.tn

[6] http ://www.bvmt.com.tn/documentation/investors/products.html

[7] http ://www.trader-finance.fr/lexique-finance

[8] http ://www.gartner.com/it-glossary/service-oriented-architecture-soa

[9] http ://www.microsoft.com/silverlight/

[10] http ://msdn.microsoft.com/fr-fr/library/ms731082.aspx

[11] http ://www.w3schools.com/html/html5_intro.asp

[12] http ://msdn.microsoft.com/en-us/library/ff647202.aspx

[13] http ://tfs.visualstudio.com/

[14] http ://msdn.microsoft.com/fr-fr/library/ms731064.aspx